

Portfolio Construction

FRE6871 & FRE7241, Fall 2025

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

December 14, 2025



NYU

**TANDON SCHOOL
OF ENGINEERING**

draft: Asset Allocation

Asset allocation means dividing an investment portfolio among different asset classes, such as large company stocks, small company stocks, international stocks, bonds, commodities, cash, etc.

The goal of asset allocation is to diversify the sources of returns and to reduce risk, depending on the investor's risk tolerance, investment goals, and investment time horizon.

For example, an investor who needs to fund college for her children might put some of her investments into government bonds that mature when her children will need to pay for college.

1,600 years ago [rabbi Isaac bar Aha](#) proposed a simple heuristic method (rule of thumb) for asset allocation: "put a third in land, a third in merchandise, and a third in cash".

```
> library(PortfolioAnalytics)
> # Use ETF returns from package rutils
> library(rutils)
> symbolv <- c("VTI", "IEF", "DBC", "XLF",
+ "VNQ", "XLP", "XLV", "XLU", "XLB", "XLE")
> # Initial portfolio to equal weights
> portf1 <- rep(1/NROW(symbolv), NROW(symbolv))
> # Named vector
> names(portf1) <- symbolv
> # Create portfolio object
> portf1 <- portfolio.spec(assets=portf1)
```

```
> # Add constraints
> portf_maxSR <- add.constraint(
+ portfolio=portf1, ## Initial portfolio
+ type="weightsum", ## Constraint sum weights
+ min_sum=0.9, max_sum=1.1)
> # Add constraints
> portf_maxSR <- add.constraint(
+ portfolio=portf_maxSR,
+ type="long_only") ## box constraint min=0, max=1
> # Add objectives
> portf_maxSR <- add.objective(
+ portfolio=portf_maxSR,
+ type="return", ## Maximize mean return
+ name="mean")
> # Add objectives
> portf_maxSR <- add.objective(
+ portfolio=portf_maxSR,
+ type="risk", ## Minimize stdev
+ name="stdev")
```

draft: Portfolio Construction

Portfolio construction means determining the amounts to be invested different assets, such as specific stocks, bonds, commodities, etc.

Portfolio optimization is one approach to portfolio construction.

Heuristic Methods for Portfolio Construction

Victor DeMiguel and others have demonstrated that optimized portfolios perform poorly out-of-sample, and that simple heuristic methods can perform better than portfolio optimization.

```
> library(PortfolioAnalytics)
> # Use ETF returns from package rutils
> library(rutils)
> symbolv <- c("VTI", "IEF", "DBC", "XLF",
+   "VNQ", "XLP", "XLV", "XLU", "XLB", "XLE")
> # Initial portfolio to equal weights
> portf1 <- rep(1/NROW(symbolv), NROW(symbolv))
> # Named vector
> names(portf1) <- symbolv
> # Create portfolio object
> portf1 <- portfolio.spec(assets=portf1)
```

```
> # Add constraints
> portf_maxSR <- add.constraint(
+   portfolio=portf1, ## Initial portfolio
+   type="weightsum", ## Constraint sum weights
+   min_sum=0.9, max_sum=1.1)
> # Add constraints
> portf_maxSR <- add.constraint(
+   portfolio=portf_maxSR,
+   type="long_only") ## box constraint min=0, max=1
> # Add objectives
> portf_maxSR <- add.objective(
+   portfolio=portf_maxSR,
+   type="return", ## Maximize mean return
+   name="mean")
> # Add objectives
> portf_maxSR <- add.objective(
+   portfolio=portf_maxSR,
+   type="risk", ## Minimize stdev
+   name="stdev")
```

Vector and Matrix Calculus

Let $\mathbf{v}$ and $\mathbf{w}$ be vectors, with $\mathbf{v} = \{v_i\}_{i=1}^{i=n}$, and let $\mathbf{1}$ be the unit vector, with $\mathbf{1} = \{1\}_{i=1}^{i=n}$.

Then the inner product of $\mathbf{v}$ and $\mathbf{w}$ can be written as $\mathbf{v}^T \mathbf{w} = \mathbf{w}^T \mathbf{v} = \sum_{i=1}^n v_i w_i$.

We can then express the sum of the elements of $\mathbf{v}$ as the inner product: $\mathbf{v}^T \mathbf{1} = \mathbf{1}^T \mathbf{v} = \sum_{i=1}^n v_i$.

And the sum of squares of $\mathbf{v}$ as the inner product: $\mathbf{v}^T \mathbf{v} = \sum_{i=1}^n v_i^2$.

Let $\mathbb{A}$ be a matrix, with $\mathbb{A} = \{A_{ij}\}_{i,j=1}^{i,j=n}$.

Then the inner product of matrix $\mathbb{A}$ with vectors $\mathbf{v}$ and $\mathbf{w}$ can be written as:

$$\mathbf{v}^T \mathbb{A} \mathbf{w} = \mathbf{w}^T \mathbb{A}^T \mathbf{v} = \sum_{i,j=1}^n A_{ij} v_i w_j$$

The derivative of a scalar variable with respect to a vector variable is a vector, for example:

$$\frac{d(\mathbf{v}^T \mathbf{1})}{d\mathbf{v}} = d_v[\mathbf{v}^T \mathbf{1}] = d_v[\mathbf{1}^T \mathbf{v}] = \mathbf{1}^T$$

$$d_v[\mathbf{v}^T \mathbf{w}] = d_v[\mathbf{w}^T \mathbf{v}] = \mathbf{w}^T$$

$$d_v[\mathbf{v}^T \mathbb{A} \mathbf{w}] = \mathbf{w}^T \mathbb{A}^T$$

$$d_v[\mathbf{v}^T \mathbb{A} \mathbf{v}] = \mathbf{v}^T \mathbb{A} + \mathbf{v}^T \mathbb{A}^T$$

The Minimum Variance Portfolio

The portfolio variance is equal to: $\mathbf{w}^T \mathbb{C} \mathbf{w}$, where $\mathbb{C}$ is the covariance matrix of returns.

If the portfolio weights $\mathbf{w}$ are subject to *linear* constraints: $\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$, then the weights that minimize the portfolio variance can be found by minimizing the *Lagrangian*:

$$\mathcal{L} = \mathbf{w}^T \mathbb{C} \mathbf{w} - \lambda (\mathbf{w}^T \mathbf{1} - 1)$$

Where λ is a *Lagrange multiplier*.

The derivative of a scalar variable with respect to a vector variable is a vector, for example:

$$d_w[\mathbf{w}^T \mathbf{1}] = d_w[\mathbf{1}^T \mathbf{w}] = \mathbf{1}^T$$

$$d_w[\mathbf{w}^T \mathbf{r}] = d_w[\mathbf{r}^T \mathbf{w}] = \mathbf{r}^T$$

$$d_w[\mathbf{w}^T \mathbb{C} \mathbf{w}] = \mathbf{w}^T \mathbb{C} + \mathbf{w}^T \mathbb{C}^T$$

Where $\mathbf{1}$ is the unit vector, and $\mathbf{w}^T \mathbf{1} = \mathbf{1}^T \mathbf{w} = \sum_{i=1}^n x_i$

The derivative of the *Lagrangian* $\mathcal{L}$ with respect to $\mathbf{w}$ is given by:

$$d_w \mathcal{L} = 2\mathbf{w}^T \mathbb{C} - \lambda \mathbf{1}^T$$

By setting the derivative to zero we find $\mathbf{w}$ equal to:

$$\mathbf{w} = \frac{1}{2} \lambda \mathbb{C}^{-1} \mathbf{1}$$

By multiplying the above from the left by $\mathbf{1}^T$, and using $\mathbf{w}^T \mathbf{1} = 1$, we find λ to be equal to:

$$\lambda = \frac{2}{\mathbf{1}^T \mathbb{C}^{-1} \mathbf{1}}$$

And finally the portfolio weights are then equal to:

$$\mathbf{w} = \frac{\mathbb{C}^{-1} \mathbf{1}}{\mathbf{1}^T \mathbb{C}^{-1} \mathbf{1}}$$

If the portfolio weights are subject to *quadratic* constraints: $\mathbf{w}^T \mathbf{w} = 1$ then the minimum variance weights are equal to the highest order *principal component* (with the smallest eigenvalue) of the covariance matrix $\mathbb{C}$.

Returns and Variance of the *Minimum Variance Portfolio*

The stock weights of the *minimum variance* portfolio under the constraint $\mathbf{w}^T \mathbf{1} = 1$ can be calculated using the inverse of the covariance matrix:

$$\mathbf{w} = \frac{\mathbf{C}^{-1} \mathbf{1}}{\mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}}$$

The daily returns of the *minimum variance* portfolio are equal to:

$$\mathbf{r}_{mv} = \frac{\mathbf{r}^T \mathbf{C}^{-1} \mathbf{1}}{\mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}} = \frac{\mathbf{r}^T \mathbf{C}^{-1} \mathbf{1}}{c_{11}}$$

Where $\mathbf{r}$ are the daily stock returns, and $c_{11} = \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}$.

The variance of the *minimum variance* portfolio is equal to:

$$\sigma_{mv}^2 = \mathbf{w}^T \mathbf{C} \mathbf{w} = \frac{\mathbf{1}^T \mathbf{C}^{-1} \mathbf{C} \mathbf{C}^{-1} \mathbf{1}}{(\mathbf{1}^T \mathbf{C}^{-1} \mathbf{1})^2} = \frac{1}{\mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}} = \frac{1}{c_{11}}$$

The function `solve()` solves systems of linear equations, and also inverts square matrices.

The `%*` operator performs *inner (scalar)* multiplication of vectors and matrices.

Inner multiplication multiplies the rows of one matrix with the columns of another matrix.

The function `drop()` removes any extra dimensions of length *one*.

```
> # Calculate daily ETF returns
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> # Calculate covariance matrix of returns and its inverse
> covmat <- cov(retp)
> covinv <- solve(a=covmat)
> unitv <- rep(1, nstocks)
> # Calculate the minimum variance weights
> c11 <- drop(t(unitv) %*% covinv %*% unitv)
> weightmv <- drop(covinv %*% unitv/c11)
> # Calculate the daily minvar portfolio returns in two ways
> retmv <- (retp %*% weightmv)
> all.equal(retmv, (retp %*% covinv %*% unitv)/c11)
> # Calculate the minimum variance in three ways
> all.equal(var(retmv),
+   t(weightmv) %*% covmat %*% weightmv,
+   1/(t(unitv) %*% covinv %*% unitv))
```

The Efficient Portfolios

A portfolio which has the smallest variance, given a target return, is an *efficient portfolio*.

The efficient portfolio weights have two constraints: the sum of portfolio weights $\mathbf{w}$ is equal to 1: $\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$, and the mean portfolio return is equal to the target return r_t : $\mathbf{w}^T \bar{\mathbf{r}} = \sum_{i=1}^n w_i \bar{r}_i = r_t$.

Where $\bar{\mathbf{r}}$ are the mean stock returns.

The stock weights that minimize the portfolio variance under these constraints can be found by minimizing the *Lagrangian*:

$$\mathcal{L} = \mathbf{w}^T \mathbf{C} \mathbf{w} - \lambda_1 (\mathbf{w}^T \mathbf{1} - 1) - \lambda_2 (\mathbf{w}^T \bar{\mathbf{r}} - r_t)$$

Where λ_1 and λ_2 are the *Lagrange multipliers*.

The derivative of the *Lagrangian* $\mathcal{L}$ with respect to $\mathbf{w}$ is given by:

$$d_{\mathbf{w}} \mathcal{L} = 2\mathbf{w}^T \mathbf{C} - \lambda_1 \mathbf{1}^T - \lambda_2 \bar{\mathbf{r}}^T$$

By setting the derivative to zero we obtain the efficient portfolio weights $\mathbf{w}$:

$$\mathbf{w} = \frac{1}{2} (\lambda_1 \mathbf{C}^{-1} \mathbf{1} + \lambda_2 \mathbf{C}^{-1} \bar{\mathbf{r}})$$

By multiplying the above from the left first by $\mathbf{1}^T$, and then by $\bar{\mathbf{r}}^T$, we obtain a system of two equations for λ_1 and λ_2 :

$$2\mathbf{1}^T \mathbf{w} = \lambda_1 \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1} + \lambda_2 \mathbf{1}^T \mathbf{C}^{-1} \bar{\mathbf{r}} = 2$$

$$2\bar{\mathbf{r}}^T \mathbf{w} = \lambda_1 \bar{\mathbf{r}}^T \mathbf{C}^{-1} \mathbf{1} + \lambda_2 \bar{\mathbf{r}}^T \mathbf{C}^{-1} \bar{\mathbf{r}} = 2r_t$$

The above can be written in matrix notation as:

$$\begin{bmatrix} \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1} & \mathbf{1}^T \mathbf{C}^{-1} \bar{\mathbf{r}} \\ \bar{\mathbf{r}}^T \mathbf{C}^{-1} \mathbf{1} & \bar{\mathbf{r}}^T \mathbf{C}^{-1} \bar{\mathbf{r}} \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 2r_t \end{bmatrix}$$

Or:

$$\begin{bmatrix} c_{11} & c_{r1} \\ c_{r1} & c_{rr} \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix} = \mathbb{F} \lambda = 2 \begin{bmatrix} 1 \\ r_t \end{bmatrix} = 2\mathbf{u}$$

With $c_{11} = \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}$, $c_{r1} = \mathbf{1}^T \mathbf{C}^{-1} \bar{\mathbf{r}}$, $c_{rr} = \bar{\mathbf{r}}^T \mathbf{C}^{-1} \bar{\mathbf{r}}$, $\lambda = \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix}$, $\mathbf{u} = \begin{bmatrix} 1 \\ r_t \end{bmatrix}$, and $\mathbb{F} = \mathbf{u}^T \mathbf{C}^{-1} \mathbf{u} = \begin{bmatrix} c_{11} & c_{r1} \\ c_{r1} & c_{rr} \end{bmatrix}$.

The *Lagrange multipliers* can be solved as:

$$\lambda = 2\mathbb{F}^{-1} \mathbf{u}$$

The Efficient Portfolio Weights

The efficient portfolio weights $\mathbf{w}$ can now be solved as:

$$\mathbf{w} = \frac{1}{2}(\lambda_1 \mathbf{C}^{-1} \mathbf{1} + \lambda_2 \mathbf{C}^{-1} \bar{\mathbf{r}}) =$$

$$\frac{1}{2} \begin{bmatrix} \mathbf{C}^{-1} \mathbf{1} \\ \mathbf{C}^{-1} \bar{\mathbf{r}} \end{bmatrix}^T \lambda = \begin{bmatrix} \mathbf{C}^{-1} \mathbf{1} \\ \mathbf{C}^{-1} \bar{\mathbf{r}} \end{bmatrix}^T \mathbb{F}^{-1} \mathbf{u} =$$

$$\frac{1}{\det \mathbb{F}} \begin{bmatrix} \mathbf{C}^{-1} \mathbf{1} \\ \mathbf{C}^{-1} \bar{\mathbf{r}} \end{bmatrix}^T \begin{bmatrix} c_{rr} & -c_{r1} \\ -c_{r1} & c_{11} \end{bmatrix} \begin{bmatrix} 1 \\ r_t \end{bmatrix} =$$

$$\frac{(c_{rr} - c_{r1} r_t) \mathbf{C}^{-1} \mathbf{1} + (c_{11} r_t - c_{r1}) \mathbf{C}^{-1} \bar{\mathbf{r}}}{\det \mathbb{F}}$$

With $c_{11} = \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}$, $c_{r1} = \mathbf{1}^T \mathbf{C}^{-1} \bar{\mathbf{r}}$, $c_{rr} = \bar{\mathbf{r}}^T \mathbf{C}^{-1} \bar{\mathbf{r}}$.
And $\det \mathbb{F} = c_{11} c_{rr} - c_{r1}^2$ is the determinant of the matrix $\mathbb{F}$.

The above formula shows that the efficient portfolio weights are a linear function of the target return.

Therefore a convex sum of two efficient portfolio weights: $w = \alpha w_1 + (1 - \alpha) w_2$, are also the weights of an *efficient portfolio*, with target return equal to:
 $r_t = \alpha r_1 + (1 - \alpha) r_2$

```
> # Calculate vector of mean returns
> retm <- colMeans(retp)
> # Specify the target return
> retarg <- 1.5*mean(retp)
> # Products of inverse with mean returns and unit vector
> c11 <- drop(t(unitv) %*% covinv %*% unitv)
> cr1 <- drop(t(unitv) %*% covinv %*% retm)
> crr <- drop(t(retm) %*% covinv %*% retm)
> fmat <- matrix(c(c11, cr1, cr1, crr), nc=2)
> # Solve for the Lagrange multipliers
> lagm <- solve(a=fmat, b=c(2, 2*retarg))
> # Calculate the efficient portfolio weights
> weightv <- 0.5*drop(covinv %*% cbind(unitv, retm) %*% lagm)
> # Calculate constraints
> all.equal(1, sum(weightv))
> all.equal(retarg, sum(retm*weightv))
```


Variance of the *Efficient Portfolios*

The *efficient portfolio* variance is equal to:

$$\sigma^2 = \mathbf{w}^T \mathbb{C} \mathbf{w} = \frac{1}{4} \lambda^T \mathbb{F} \lambda = \mathbf{u}^T \mathbb{F}^{-1} \mathbf{u} =$$

$$\frac{1}{\det \mathbb{F}} \begin{bmatrix} 1 \\ r_t \end{bmatrix}^T \begin{bmatrix} c_{rr} & -c_{r1} \\ -c_{r1} & c_{11} \end{bmatrix} \begin{bmatrix} 1 \\ r_t \end{bmatrix} =$$

$$\frac{c_{11}r_t^2 - 2c_{r1}r_t + c_{rr}}{\det \mathbb{F}}$$

The above formula shows that the variance of the *efficient portfolios* is a *parabola* with respect to the target return r_t .

The vertex of the *parabola* is the minimum variance portfolio: $r_{mv} = c_{r1}/c_{11} = \mathbf{1}^T \mathbb{C}^{-1} \mathbf{r} / \mathbf{1}^T \mathbb{C}^{-1} \mathbf{1}$ and $\sigma_{mv}^2 = 1/c_{11} = 1/\mathbf{1}^T \mathbb{C}^{-1} \mathbf{1}$.

The *efficient portfolio* variance can be expressed in terms of the difference $\Delta_r = r_t - r_{mv}$ as:

$$\sigma^2 = \frac{\Delta_r^2 + \det \mathbb{F}}{c_{11} \det \mathbb{F}}$$

So that if $\Delta_r = 0$ then $\sigma^2 = 1/c_{11}$.

Where $\det \mathbb{F} = c_{11}c_{rr} - c_{r1}^2$ is the determinant of the matrix $\mathbb{F}$.

```
> # Calculate the efficient portfolio returns
> reteff <- drop(retp %*% weightv)
> reteffm <- mean(reteff)
> all.equal(reteffm, retarg)
> # Calculate the efficient portfolio variance in three ways
> uu <- c(1, retarg)
> finv <- solve(fmat)
> detf <- (c11*crr-cr1^2) ## det(fmat)
> all.equal(var(reteff),
+ drop(t(uu) %*% finv %*% uu),
+ (c11*reteffm^2-2*cr1*reteffm+crr)/detf)
```

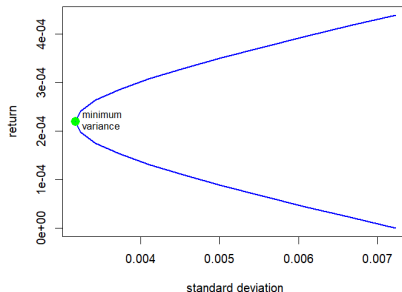
The Efficient Frontier

The *efficient frontier* is the set of *efficient portfolios*, that have the lowest risk (standard deviation) for the given level of return.

The *efficient frontier* is the plot of the target returns r_t and the standard deviations of the *efficient portfolios*, which is a *hyperbola*.

```
> # Calculate the daily and mean minvar portfolio returns
> c11 <- drop(t(unitv) %*% covinv %*% unitv)
> weightv <- drop(covinv %*% unitv/c11)
> retmv <- (retp %*% weightv)
> retmvm <- sum(weightv*retp)
> # Calculate the minimum variance
> varmv <- 1/c11
> stdevmv <- sqrt(varmv)
> # Calculate efficient frontier from target returns
> retargv <- retmvm*(1+seq(from=(-1), to=1, by=0.1))
> stdevs <- sapply(retargv, function(rett) {
+   uu <- c(1, rett)
+   sqrt(drop(t(uu) %*% finv %*% uu))
+ }) ## end sapply
```

Efficient Frontier and Minimum Variance Portfolio



```
> # Plot the efficient frontier
> plot(x=stdevs, y=retargv, t="l", col="blue", lwd=2,
+   main="Efficient Frontier and Minimum Variance Portfolio",
+   xlab="standard deviation", ylab="return")
> points(x=stdevmv, y=retmvm, col="green", lwd=6)
> text(x=stdevmv, y=retmvm, labels="minimum \nvariance",
+   pos=4, cex=0.8)
```

The Tangent Line and the Risk-free Rate

The *tangent* line connects the risk-free point ($\sigma = 0, r = r_f$) with a single tangent point on the *efficient frontier*.

A *tangent* line can be drawn at every point on the *efficient frontier*.

The slope β of the *tangent* line can be calculated by differentiating the efficient portfolio variance σ^2 by the target return r_t :

$$\begin{aligned}\frac{d\sigma^2}{dr_t} &= 2\sigma \frac{d\sigma}{dr_t} = \frac{2c_{11}r_t - 2c_{r1}}{\det \mathbb{F}} \\ \frac{d\sigma}{dr_t} &= \frac{c_{11}r_t - c_{r1}}{\sigma \det \mathbb{F}} \\ \beta &= \frac{\sigma \det \mathbb{F}}{c_{11}r_t - c_{r1}}\end{aligned}$$

The *tangent* line connects the *tangent* point on the *efficient frontier* with a *risk-free* rate r_f .

The *risk-free* rate r_f can be calculated as the intercept of the tangent line:

$$\begin{aligned}r_f &= r_t - \sigma \beta = r_t - \frac{\sigma^2 \det \mathbb{F}}{c_{11}r_t - c_{r1}} = \\ r_t - \frac{c_{11}r_t^2 - 2c_{r1}r_t + c_{rr}}{\det \mathbb{F}} \frac{\det \mathbb{F}}{c_{11}r_t - c_{r1}} &= \\ r_t - \frac{c_{11}r_t^2 - 2c_{r1}r_t + c_{rr}}{c_{11}r_t - c_{r1}} &= \frac{c_{r1}r_t - c_{rr}}{c_{11}r_t - c_{r1}}\end{aligned}$$

```
> # Calculate standard deviation of efficient portfolio
> uu <- c(1, retarg)
> stdeveff <- sqrt(drop(t(uu) %*% finv %*% uu))
> # Calculate the slope of the tangent line
> detf <- (c11*crr - cr1^2) ## det(fmat)
> sharper <- (stdeveff*detf)/(c11*retarg - cr1)
> # Calculate the risk-free rate as intercept of the tangent line
> raterf <- retarg - sharper*stdeveff
> # Calculate the risk-free rate from target return
> all.equal(raterf,
+ (retarg*cr1 - crr)/(retarg*c11 - cr1))
```

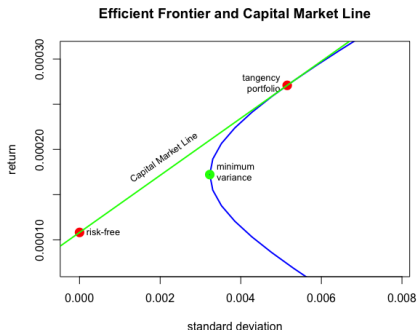
The Capital Market Line

The *Capital Market Line* (CML) is the tangent line connecting the risk-free point ($\sigma = 0, r = r_f$) with a single tangent point on the *efficient frontier*.

The *tangency portfolio* is the *efficient portfolio* at the tangent point corresponding to the given *risk-free rate*.

Each value of the *risk-free rate* r_f corresponds to a unique *tangency portfolio*.

For a given *risk-free rate* r_f , the *tangency portfolio* has the highest *Sharpe ratio* among all the *efficient portfolios*.



```
> # Plot efficient frontier
> aspectr <- 1.0*max(stdevs)/diff(range(retargv)) ## Aspect ratio
> plot(x=stdevs, y=retargv, t="l", col="blue", lwd=2, asp=aspectr,
+      xlim=c(0.4, 0.6)*max(stdevs), ylim=c(0.2, 0.9)*max(retargv),
+      main="Efficient Frontier and Capital Market Line",
+      xlab="standard deviation", ylab="return")
> # Plot the minimum variance portfolio
> points(x=stdevmv, y=retmv, col="green", lwd=6)
> text(x=stdevmv, y=retmv, labels="minimum \nvariance",
+      pos=4, cex=0.8)
```

```
> # Plot the tangency portfolio
> points(x=stdeveff, y=retarg, col="red", lwd=6)
> text(x=stdeveff, y=retarg, labels="tangency\nportfolio", pos=2, cex=0.8)
> # Plot the risk-free point
> points(x=0, y=raterf, col="red", lwd=6)
> text(x=0, y=raterf, labels="risk-free", pos=4, cex=0.8)
> # Plot the tangent line
> abline(a=raterf, b=sharper, lwd=2, col="green")
> text(x=0.6*stdev, y=0.8*retarg,
+      labels="Capital Market Line", pos=2, cex=0.8,
+      srt=180/pi*atan(aspectr*sharper))
```

The Capital Market Line Portfolios

The points on the *Capital Market Line* represent portfolios consisting of the *tangency portfolio* and the *risk-free asset* (bond).

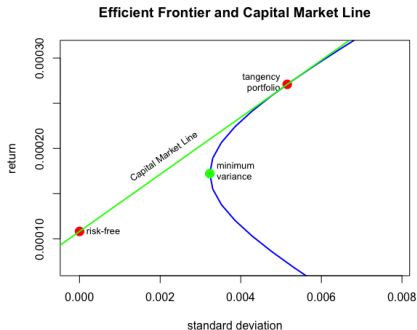
The *Capital Market Line* represents delevered and levered portfolios, consisting of the *tangency portfolio* combined with the *risk-free asset* (bond).

The *CML* portfolios have weights proportional to the tangency portfolio weights.

The *CML* portfolios above the tangent point are levered with respect to the *tangency portfolio* through borrowing at the *risk-free rate* r_f . Their weights are equal to the tangency portfolio weights multiplied by a factor greater than 1.

The *CML* portfolios below the tangent point are delevered with respect to the *tangency portfolio* through investing at the *risk-free rate* r_f . Their weights are equal to the tangency portfolio weights multiplied by a factor less than 1.

All the *CML* portfolios have the same *Sharpe ratio*.



Maximum Sharpe Portfolio Weights

The *Sharpe ratio* is equal to the ratio of excess returns divided by the portfolio standard deviation:

$$SR = \frac{\mathbf{w}^T \boldsymbol{\mu}}{\sigma}$$

Where $\boldsymbol{\mu} = \bar{\mathbf{r}} - r_f$ is the vector of mean excess returns (in excess of the risk-free rate r_f), $\mathbf{w}$ is the vector of portfolio weights, and $\sigma = \sqrt{\mathbf{w}^T \mathbb{C} \mathbf{w}}$, where $\mathbb{C}$ is the covariance matrix of returns.

We can calculate the *maximum Sharpe* portfolio weights by setting the derivative of the *Sharpe ratio* with respect to the weights $\mathbf{w}$ to zero:

$$d_w SR = \frac{1}{\sigma} \left(\boldsymbol{\mu}^T - \frac{(\mathbf{w}^T \boldsymbol{\mu})(\mathbf{w}^T \mathbb{C})}{\sigma^2} \right) = 0$$

We then get:

$$(\mathbf{w}^T \mathbb{C} \mathbf{w}) \boldsymbol{\mu} = (\mathbf{w}^T \boldsymbol{\mu}) \mathbb{C} \mathbf{w}$$

We can multiply the above equation by $\mathbb{C}^{-1}$ to get:

$$\mathbf{w} = \frac{\mathbf{w}^T \mathbb{C} \mathbf{w}}{\mathbf{w}^T \boldsymbol{\mu}} \mathbb{C}^{-1} \boldsymbol{\mu}$$

We can finally rescale the weights so that they satisfy the linear constraint $\mathbf{w}^T \mathbf{1} = 1$:

$$\mathbf{w} = \frac{\mathbb{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbb{C}^{-1} \boldsymbol{\mu}}$$

These are the weights of the *maximum Sharpe* portfolio, with the vector of mean excess returns equal to $\boldsymbol{\mu}$, and the covariance matrix equal to $\mathbb{C}$.

The *maximum Sharpe* portfolio is an *efficient portfolio*, and so its mean return is equal to some target return r_t : $\bar{\mathbf{r}}^T \mathbf{w} = \sum_{i=1}^n w_i r_i = r_t$.

The mean return of the *maximum Sharpe* portfolio is equal to:

$$r_t = \bar{\mathbf{r}}^T \mathbf{w} = \frac{\bar{\mathbf{r}}^T \mathbb{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbb{C}^{-1} \boldsymbol{\mu}} = \frac{\bar{\mathbf{r}}^T \mathbb{C}^{-1} (\bar{\mathbf{r}} - r_f)}{\mathbf{1}^T \mathbb{C}^{-1} (\bar{\mathbf{r}} - r_f)} = \frac{\bar{\mathbf{r}}^T \mathbb{C}^{-1} \mathbf{1} r_f - \bar{\mathbf{r}}^T \mathbb{C}^{-1} \mathbf{1} \bar{\mathbf{r}}}{\mathbf{1}^T \mathbb{C}^{-1} \mathbf{1} r_f - \bar{\mathbf{r}}^T \mathbb{C}^{-1} \mathbf{1}} = \frac{c_{r1} r_f - c_{rr}}{c_{11} r_f - c_{r1}}$$

The above formula calculates the target return r_t from the risk-free rate r_f .

Returns and Variance of the Maximum Sharpe Portfolio

The *maximum Sharpe* portfolio weights depend on the value of the risk-free rate r_f :

$$\mathbf{w} = \frac{\mathbf{C}^{-1}\boldsymbol{\mu}}{\mathbf{1}^T\mathbf{C}^{-1}\boldsymbol{\mu}} = \frac{\mathbf{C}^{-1}(\bar{\mathbf{r}} - r_f)}{\mathbf{1}^T\mathbf{C}^{-1}(\bar{\mathbf{r}} - r_f)}$$

The mean return of the *maximum Sharpe* portfolio is equal to:

$$r_t = \bar{\mathbf{r}}^T \mathbf{w} = \frac{\bar{\mathbf{r}}^T \mathbf{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu}} = \frac{c_{r1} r_f - c_{rr}}{c_{11} r_f - c_{r1}}$$

The variance of the *maximum Sharpe* portfolio is equal to:

$$\begin{aligned} \sigma^2 &= \mathbf{w}^T \mathbf{C} \mathbf{w} = \frac{\boldsymbol{\mu}^T \mathbf{C}^{-1} \mathbf{C} \mathbf{C}^{-1} \boldsymbol{\mu}}{(\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu})^2} = \frac{\boldsymbol{\mu}^T \mathbf{C}^{-1} \boldsymbol{\mu}}{(\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu})^2} = \\ &= \frac{(\bar{\mathbf{r}} - r_f)^T \mathbf{C}^{-1} (\bar{\mathbf{r}} - r_f)}{(\mathbf{1}^T \mathbf{C}^{-1} (\bar{\mathbf{r}} - r_f))^2} = \frac{c_{11} r_t^2 - 2c_{r1} r_t + c_{rr}}{\det \mathbb{F}} \end{aligned}$$

The above formula expresses the *maximum Sharpe* portfolio variance as a function of its mean return r_t .

The *maximum Sharpe* ratio is equal to:

$$\begin{aligned} SR &= \frac{\mathbf{w}^T \boldsymbol{\mu}}{\sigma} = \frac{\boldsymbol{\mu}^T \mathbf{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu}} / \frac{\sqrt{\boldsymbol{\mu}^T \mathbf{C}^{-1} \boldsymbol{\mu}}}{\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu}} = \\ &= \frac{\sqrt{\boldsymbol{\mu}^T \mathbf{C}^{-1} \boldsymbol{\mu}}}{\sqrt{(\bar{\mathbf{r}} - r_f)^T \mathbf{C}^{-1} (\bar{\mathbf{r}} - r_f)}} \end{aligned}$$

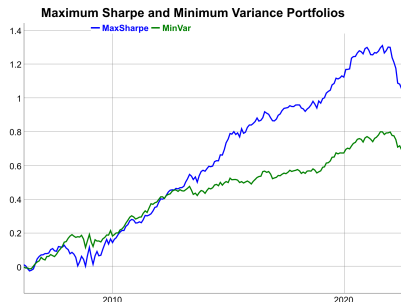
```
> # Calculate the mean excess returns
> raterf <- retarg - sharper*stdevdiff
> retx <- (retm - raterf)
> # Calculate the efficient portfolio weights
> weightv <- 0.5*drop(covinv %*% cbind(unitv, retm) %*% lagm)
> # Calculate the maximum Sharpe weights
> weightms <- drop(covinv %*% retx)/sum(covinv %*% retx)
> all.equal(weightv, weightms)
> # Calculate the maximum Sharpe mean return in two ways
> all.equal(sum(retm*weightv), (c11*raterf-crr)/(c11*raterf-cr1))
> # Calculate the maximum Sharpe daily returns
> rettd <- (retp %*% weightms)
> # Calculate the maximum Sharpe variance in four ways
> detf <- (c11*crr-cr1^2) ## det(fmat)
> all.equal(var(rettd),
+   t(weightv) %*% covmat %*% weightv,
+   (t(retx) %*% covinv %*% retx)/sum(covinv %*% retx)^2,
+   (c11*retarg^2-2*c11*retarg*crr)/detf)
> # Calculate the maximum Sharpe ratio
> sqrt(252)*sum(weightv*retx)/
+   sqrt(drop(t(weightv) %*% covmat %*% weightv))
> # Calculate the stock Sharpe ratios
> sqrt(252)*sapply((retp - raterf), function(x) mean(x)/sd(x))
```

Maximum Sharpe and Minimum Variance Performance

The *maximum Sharpe* and *Minimum Variance* portfolios are both *efficient portfolios*, with the lowest risk (standard deviation) for the given level of return.

The *maximum Sharpe* portfolio has both a higher Sharpe ratio and higher absolute returns.

```
> # Calculate optimal portfolio returns
> wealthv <- cbind(retp %*% weightms, retp %*% weightmv)
> wealthv <- xts::xts(wealthv, zoo::index(retp))
> colnames(wealthv) <- c("MaxSharpe", "MinVar")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   (mean(x)-raterf)/c(Sharpe=sd(x), Sortino=sd(x[x<0])))
> # Plot the log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Maximum Sharpe and Minimum Variance Portfolios") %>%
+   dyOptions(colors=c("blue", "green"), strokeWidth=2) %>%
+   dyLegend(show="always", width=500)
```



draft: Maximum Sharpe Portfolio of Two Stocks

Given the covariance matrix of two stocks:

$$\mathbb{C} = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{bmatrix}$$

The inverse of the covariance matrix is equal to:

$$\mathbb{C}^{-1} = \frac{1}{(\sigma_1^2 \sigma_2^2 - \sigma_{12}^2)} \begin{bmatrix} \sigma_2^2 & -\sigma_{12} \\ -\sigma_{12} & \sigma_1^2 \end{bmatrix}$$

Where σ_1^2 and σ_2^2 are the stock variances and σ_{12} is their covariance.

The *maximum Sharpe* portfolio weights are equal to:

$$\mathbf{w} = \frac{\mathbb{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbb{C}^{-1} \boldsymbol{\mu}} = \frac{1}{D} \begin{bmatrix} \sigma_2^2 \mu_1 - \sigma_{12} \mu_2 \\ -\sigma_{12} \mu_1 + \sigma_1^2 \mu_2 \end{bmatrix}$$

Where μ_1 and μ_2 are the mean excess returns of the two stocks, and $D = \sigma_2^2 \mu_1 + \sigma_1^2 \mu_2 - \sigma_{12}(\mu_1 + \mu_2)$.

```
> # Calculate the covariance, variances and the mean returns
> retp <- na.omit(rutils::etfenv$returns[, c("VTI", "TLT")])
> retew <- 0.4*retp[, 1] + 0.6*retp[, 2]
> covm <- cov(retp)
> covv <- covm[1, 2]
> var1 <- covm[1, 1]
> var2 <- covm[2, 2]
> ret1 <- mean(retp[, 1])
> ret2 <- mean(retp[, 2])
> # Calculate the inverse covariance matrix
> scalev <- (var1*var2 - covv^2)
> covinv <- matrix(c(var2, -covv, -covv, var1), nrow=2)/scalev
> round(covinv %*% covm, 6)
> # Calculate the maximum Sharpe portfolio weights
> ww <- (covinv %*% c(ret1, ret2))
> # Or
> (var2*ret1 - covv*ret2)/scalev
> (-covv*ret1 + var1*ret2)/scalev
> # Calculate the maximum Sharpe portfolio pnls
> pnls <- retp %*% ww
> pnls <- sd(retew)/sd(pnls)*pnls
> # Calculate the Sharpe ratios
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("Balanced", "MaxSharpe")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of combined log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Wealth of Fixed Share and Equal Wealth Portfolios") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

draft: Recursive Matrix Inverse

The inverse of a square matrix $\mathbb{A}$ can be calculated approximately using the recursive *Schulz formula*:

$$\mathbb{A}_{i+1}^{-1} = 2\mathbb{A}_i^{-1} - \mathbb{A}_i^{-1}\mathbb{A}\mathbb{A}_i^{-1}$$

The *Schulz formula* requires a good initial value for the inverse matrix $\mathbb{A}_1^{-1}$ or else the recursion diverges.

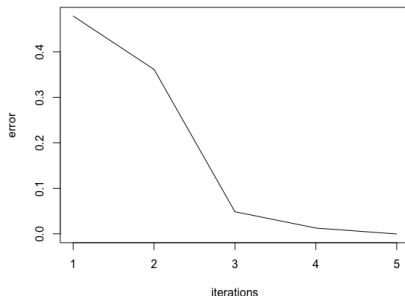
If the initial inverse matrix $\mathbb{A}_1^{-1}$ is very close to the actual inverse $\mathbb{A}^{-1}$, then the *Schulz formula* produces a very good approximation with just a few iterations.

The *Schulz formula* is useful for updating the inverse when the matrix $\mathbb{A}$ changes only slightly. For example, for updating the inverse of the covariance matrix as it changes slowly over time.

The super-assignment operator "<<=" modifies variables in the *enclosing* environment in which the function was *defined* (*lexical scoping*).

```
> # Create a random matrix
> matv <- matrix(rnorm(100), nc=10)
> # Calculate the inverse of matv
> invmat <- solve(a=matv)
> # Multiply inverse with matrix
> round(invmat %*% matv, 4)
> # Calculate the initial inverse
> invmatr <- invmat + matrix(rnorm(100, sd=0.1), nc=10)
> # Calculate the approximate recursive inverse of matv
> invmatr <- (2*invmatr - invmatr %*% matv %*% invmatr)
> # Calculate the sum of the off-diagonal elements
> sum((invmatr %*% matv)[upper.tri(matv)])
```

Iterations of Recursive Matrix Inverse



```
> # Calculate the recursive inverse of matv in a loop
> invmatr <- invmat + matrix(rnorm(100, sd=0.1), nc=10)
> iterv <- sapply(1:5, function(x) {
+   # Calculate the recursive inverse of matv
+   invmatr <<- (2*invmatr - invmatr %*% matv %*% invmatr)
+   # Calculate the sum of the off-diagonal elements
+   sum((invmatr %*% matv)[upper.tri(matv)])
+ }) ## end sapply
> # Plot the iterations
> plot(x=1:5, y=iterv, t="l", xlab="iterations", ylab="error",
+      main="Iterations of Recursive Matrix Inverse")
```

The Maximum Sharpe Portfolios and the Efficient Frontier

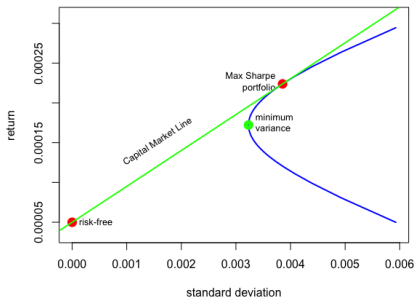
The *maximum Sharpe* portfolios are *efficient portfolios*, so they form the *efficient frontier*.

A *market portfolio* is the portfolio of all the available assets, with weights proportional to their market capitalizations.

The *maximum Sharpe* portfolio is sometimes considered to be the *market portfolio*, because it's the optimal portfolio for the given value of the risk-free rate r_f .

```
> # Calculate the maximum Sharpe portfolios for different risk-free
> detf <- (c11*crr-cr1^2) ## det(fmat)
> raterfv <- retmvm*seq(from=1.3, to=20, by=0.1)
> raterfv <- c(raterfv, retmvm*seq(from=(-20), to=0.7, by=0.1))
> effront <- sapply(raterfv, function(raterf) {
+   ## Calculate the maximum Sharpe mean return
+   reteffm <- (cr1*raterf-crr)/(c11*raterf-cr1)
+   ## Calculate the maximum Sharpe standard deviation
+   stdev <- sqrt((c11*reteffm^2-2*cr1*reteffm+crr)/detf)
+   c(return=reteffm, stdev=stdev)
+ }) ## end sapply
> effront <- effront[, order(effront["return", ])]
> # Plot the efficient frontier
> reteffv <- effront["return", ]
> stdevs <- effront["stdev", ]
> aspectr <- 0.6*max(stdevs)/diff(range(reteffv)) ## Aspect ratio
> plot(x=stdevs, y=reteffv, t="l", col="blue", lwd=2, asp=aspectr,
+   main="Maximum Sharpe Portfolio and Efficient Frontier",
+   xlim=c(0.0, max(stdevs)), xlab="standard deviation", ylab="return")
> # Plot the minimum variance portfolio
> points(x=stdevmv, y=retmvm, col="green", lwd=6)
> text(x=stdevmv, y=retmvm, labels="minimum \nvariance", pos=4, ce=
```

Maximum Sharpe Portfolio and Efficient Frontier



```
> # Calculate the maximum Sharpe return and standard deviation
> raterf <- min(reteffv)
> retmax <- (cr1*raterf-crr)/(c11*raterf-cr1)
> stdevmax <- sqrt((c11*retmax^2-2*cr1*retmax+crr)/detf)
> # Plot the maximum Sharpe portfolio
> points(x=stdevmax, y=retmax, col="red", lwd=6)
> text(x=stdevmax, y=retmax, labels="Max Sharpe\nportfolio", pos=2,
> # Plot the risk-free point
> points(x=0, y=raterf, col="red", lwd=6)
> text(x=0, y=raterf, labels="risk-free", pos=4, cex=0.8)
> # Plot the tangent line
> sharper <- (stdevmax*detf)/(c11*retmax-cr1)
> abline(a=raterf, b=sharper, lwd=2, col="green")
> text(x=0.6*stdevmax, y=0.8*retmax, labels="Capital Market Line",
+   pos=2, cex=0.8, srt=180/pi*atan(aspectr*sharper))
```

Efficient Portfolios and Their Tangent Lines

The *efficient frontier* consists of all the *maximum Sharpe* portfolios corresponding to different values of the risk-free rate.

The target return can be expressed as a function of the risk-free rate as:

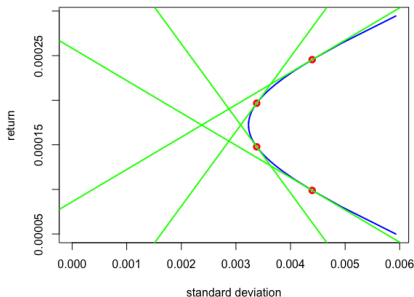
$$r_t = \frac{C_{r1} r_f - C_{rr}}{C_{l1} r_f - C_{r1}}$$

If $r_f \rightarrow \pm\infty$ then $r_t \rightarrow r_{mv} = C_{r1}/C_{l1}$.

But if the risk-free rate tends to the mean returns of the minimum variance portfolio: $r_f \rightarrow r_{mv} = C_{r1}/C_{l1}$, then $r_t \rightarrow \pm\infty$, which means that there is no efficient portfolio corresponding to the risk-free rate equal to the mean returns of the minimum variance portfolio: $r_f = r_{mv} = C_{r1}/C_{l1}$.

```
> # Plot the efficient frontier
> reteffv <- efffront["return", ]
> stdevs <- efffront["stdev", ]
> plot(x=stdevs, y=reteffv, t="l", col="blue", lwd=2,
+      xlim=c(0.0, max(stdevs)),
+      main="Efficient Frontier and Tangent Lines",
+      xlab="standard deviation", ylab="return")
```

Efficient Frontier and Tangent Lines

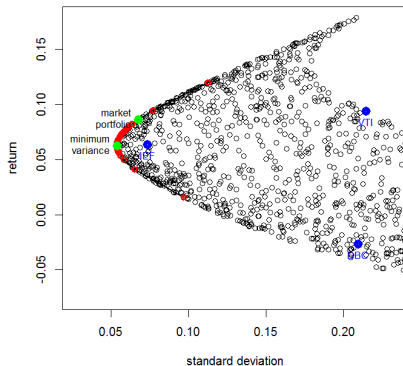


```
> # Calculate vector of mean returns
> reteffv <- min(reteffv) + diff(range(reteffv))*c(0.2, 0.4, 0.6, 0.8)
> # Plot the tangent lines
> for (reteffm in reteffv) {
+   ## Calculate the maximum Sharpe standard deviation
+   stdev <- sqrt((c11*reteffm^2-2*c11*reteffm+crr)/detf)
+   ## Calculate the slope of the tangent line
+   sharper <- (stdev*detf)/(c11*reteffm-c11)
+   ## Calculate the risk-free rate as intercept of the tangent line
+   raterf <- reteffm - sharper*stdev
+   ## Plot the tangent portfolio
+   points(x=stdev, y=reteffm, col="red", lwd=3)
+   ## Plot the tangent line
+   abline(a=raterf, b=sharper, lwd=2, col="green")
+ } ## end for
```

Random Portfolios

```
> # Calculate random portfolios
> nportf <- 1000
> randportf <- sapply(1:nportf, function(it) {
+   weightv <- runif(nstocks-1, min=-0.25, max=1.0)
+   weightv <- c(weightv, 1-sum(weightv))
+   ## Portfolio returns and standard deviation
+   c(return=252*sum(weightv*retm),
+     stdev=sqrt(252*drop(weightv %%% covmat %%% weightv)))
+ }) ## end sapply
> # Plot scatterplot of random portfolios
> x11(widthp <- 6, heightp <- 6)
> plot(x=randportf["stdev", ], y=randportf["return", ],
+   main="Efficient Frontier and Random Portfolios",
+   xlim=c(0.5*stdev, 0.8*max(randportf["stdev", ])),
+   xlab="standard deviation", ylab="return")
> # Plot maximum Sharpe portfolios
> lines(x=effront[, "stdev"], y=effront[, "return"], lwd=2)
> points(x=effront[, "stdev"], y=effront[, "return"],
+   col="red", lwd=3)
> # Plot the minimum variance portfolio
> points(x=stdev, y=retp, col="green", lwd=6)
> text(stdev, retp, labels="minimum\nvariance", pos=2, cex=0.8)
> # Plot efficient portfolio
> points(x=effront[marketp, "stdev"],
+   y=effront[marketp, "return"], col="green", lwd=6)
> text(x=effront[marketp, "stdev"], y=effront[marketp, "return"],
+   labels="market\nportfolio", pos=2, cex=0.8)
```

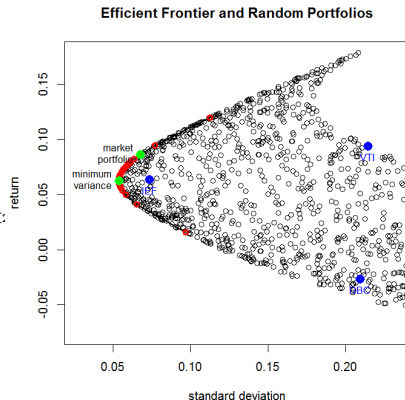
Efficient Frontier and Random Portfolios



```
> # Plot individual assets
> points(x=sqrt(252*diag(covmat)),
+   y=252*retm, col="blue", lwd=6)
> text(x=sqrt(252*diag(covmat)), y=252*retm,
+   labels=names(retm),
+   col="blue", pos=1, cex=0.8)
```

draft: Plotting Random Portfolios Without Using Covariance Matrix

```
> # Vector of symbol names
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> # Calculate random portfolios
> nportf <- 1000
> randportf <- sapply(1:nportf, function(it) {
+   weightv <- runif(nstocks, min=0, max=10)
+   weightv <- weightv/sum(weightv)
+   retp <- rutils::etfenv$returns[, symbolv] %*% weightv
+   100*c(ret=mean(retp), sd=sd(retp))
+ }) ## end sapply
> # Plot scatterplot of random portfolios
> x11(width=6, height=5)
> plot(x=randportf[2, ], y=randportf[1, ], xlim=c(0, max(randportf[2, ])),
+   main="Random portfolios",
+   ylim=c(min(0, min(randportf[1, ])), max(randportf[1, ])),
+   xlab=rownames(randportf)[2], ylab=rownames(randportf)[1])
```



```
> # Plot individual assets
> points(x=sqrt(252*diag(covmat)),
+   y=252*retm, col="blue", lwd=6)
> text(x=sqrt(252*diag(covmat)), y=252*retm,
+   labels=names(retm),
+   col="blue", pos=1, cex=0.8)
```

draft: Efficient Frontier for Two-stock Portfolio

The covariance matrix for two stocks is equal to:

$$\mathbb{C} = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{pmatrix}$$

Where σ_{11} is the variance, and σ_{12} is the covariance of returns between the two stocks.

The excess returns of a two-stock portfolio are equal to:

Any linear combination of a portfolio of two stocks is a maximum Sharpe portfolio corresponding to some value of the risk-free rate.

$$R = w\mu_1 + (1 - w)\mu_2$$

Solving for the weight w :

$$w = (R - \mu_2) / (\mu_1 - \mu_2)$$

The variance of the *maximum Sharpe* portfolio is equal to:

$$\sigma^2 = \frac{\mu^T \mathbb{C}^{-1} \mathbb{C} \mathbb{C}^{-1} \mu}{(\mathbf{1}^T \mathbb{C}^{-1} \mu)^2} = \frac{\mu^T \mathbb{C}^{-1} \mu}{(\mathbf{1}^T \mathbb{C}^{-1} \mu)^2}$$

If the correlations of returns are equal to zero, then The *minimum variance* portfolio weights are proportional to the inverse of the individual variances:

$$w_i = \frac{1}{\sigma_i^2 \sum_{i=1}^n \sigma_i^{-2}}$$

The *maximum Sharpe* portfolio weights are proportional to the ratio of excess returns divided by the individual variances:

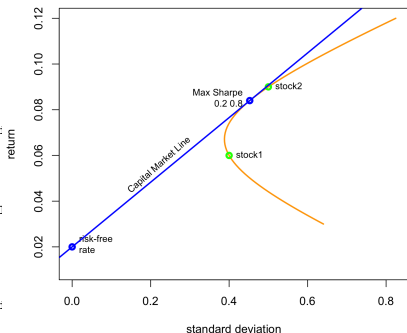
$$w_i = \frac{\mu_i}{\sigma_i^2 \sum_{i=1}^n \mu_i \sigma_i^{-2}}$$

Efficient Frontier for Two-stock Portfolio

The *efficient frontier* for a two-stock portfolio illustrates its dependence on the correlation and the risk-free rate.

```
> # Define the parameters
> raterf <- 0.02 ## Risk-free rate
> retp <- c(stock1=0.06, stock2=0.09) ## Returns
> stdevs <- c(stock1=0.4, stock2=0.5) ## Standard deviations
> corrp <- 0.6 ## Correlation
> covmat <- matrix(c(1, corrp, corrp, 1), nc=2) ## Covariance matrix
> covmat <- t(t(stdevs*covmat)*stdevs)
> weightv <- seq(from=-1, to=2, length.out=71) ## Weights
> weightv <- cbind(weightv, 1-weightv)
> retport <- weightv %*% retp ## Portfolio returns
> portfsd <- sqrt(rowSums(weightv*(weightv %*% covmat))) ## Portfolio standard deviation
> sharper <- (retport-raterf)/portfsd ## Portfolio Sharpe ratios
> # Plot the efficient frontier
> # x11(widthhp <- 6, heightp <- 5) ## Windows
> dev.new(widthhp <- 6, heightp <- 5, noRStudioGD=TRUE) ## Mac
> plot(portfsd, retport, t="l",
+   main=paste0("Efficient Frontier and CML for Two Stocks\ncorrelat:", corrp),
+   xlab="standard deviation", ylab="return",
+   lwd=2, col="orange", xlim=c(0, max(portfsd)), ylim=c(0.01, max(retport)),
+   # Add the maximum Sharpe portfolio
+   whichmax <- which.max(sharper)
+   sharperm <- max(sharper) ## Maximum Sharpe ratio
+   retmax <- retport[whichmax]
+   sdeff <- portfsd[whichmax]
+   weightm <- round(weightv[whichmax], 2)
+   points(sdeff, retmax, col="blue", lwd=3)
+   text(x=sdeff, y=retmax, labels=paste(c("Max Sharpe\n",
+     structure(c(weightm, (1-weightm)), names=c("stock1", "stock2")),
+     pos=2, cex=0.8)
```

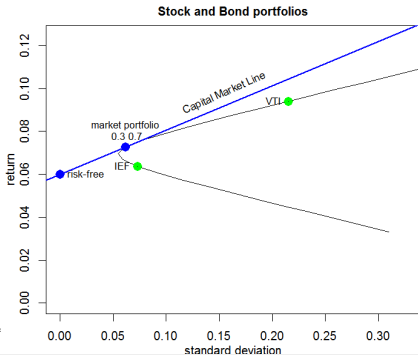
Efficient Frontier and CML for Two Stocks
correlation = 60%



```
> # Plot individual stocks
> points(stdevs, retp, col="green", lwd=3)
> text(stdevs, retp, labels=names(retp), pos=4, cex=0.8)
> # Add point at risk-free rate and draw Capital Market Line
> points(x=0, y=raterf, col="blue", lwd=3)
> text(0, raterf, labels="risk-free\nrate", pos=4, cex=0.8)
> abline(a=raterf, b=sharperm, lwd=2, col="blue")
> rangev <- par("usr")
> text(sdeff/2, (retmax+raterf)/2,
+   labels="Capital Market Line", cex=0.8, , pos=3,
+   srt=45*atan(sharperm*(rangev[2]-rangev[1])/
+     (rangev[4]-rangev[3])*heightp/widthp)/(0.25*pi))
```


Efficient Frontier of Stock and Bond Portfolios

```
> # Vector of symbol names
> symbolv <- c("VTI", "IEF")
> # Matrix of portfolio weights
> weightv <- seq(from=-1, to=2, length.out=31)
> weightv <- cbind(weightv, 1-weightv)
> # Calculate portfolio returns and volatilities
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> retport <- retp %>% t(weightv)
> portfv <- cbind(252*colMeans(retport),
+   sqrt(252)*matrixStats::colSDs(retport))
> colnames(portfv) <- c("returns", "stdev")
> raterf <- 0.06
> portfv <- cbind(portfv,
+   (portfv[, "returns"]-raterf)/portfv[, "stdev"])
> colnames(portfv)[3] <- "Sharpe"
> whichmax <- which.max(portfv[, "Sharpe"])
> sharpem <- portfv[whichmax, "Sharpe"]
> plot(x=portfv[, "stdev"], y=portfv[, "returns"],
+   main="Stock and Bond portfolios", t="l",
+   xlim=c(0, 0.7*max(portfv[, "stdev"])), ylim=c(0, max(portfv[,
+   "returns"])), xlab="standard deviation", ylab="return")
> # Add blue point for efficient portfolio
> points(x=portfv[whichmax, "stdev"], y=portfv[whichmax, "returns"])
> text(x=portfv[whichmax, "stdev"], y=portfv[whichmax, "returns"],
+   labels=paste(c("efficient portfolio\n",
+   structure(c(weightv[whichmax, 1], weightv[whichmax, 2]), names=
+   pos=3, cex=0.8)
```



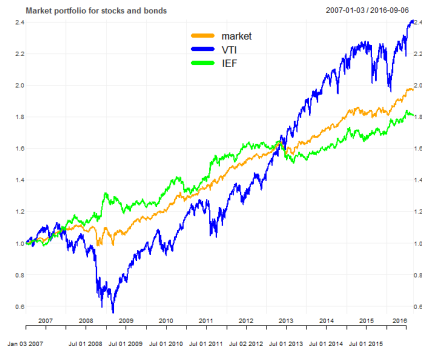
```
> # Plot individual stocks
> retm <- 252*sapply(retport, mean)
> stdevs <- sqrt(252)*sapply(retport, sd)
> points(stdevs, retm, col="green", lwd=6)
> text(stdevs, retm, labels=names(retport), pos=2, cex=0.8)
> # Add point at risk-free rate and draw Capital Market Line
> points(x=0, y=raterf, col="blue", lwd=6)
> text(0, raterf, labels="risk-free", pos=4, cex=0.8)
> abline(a=raterf, b=sharpem, col="blue", lwd=2)
> rangev <- par("usr")
> text(max(portfv[, "stdev"])/3, 0.75*max(portfv[, "returns"]),
+   labels="Capital Market Line", cex=0.8, , pos=3,
+   srt=45*atan(sharpem*(rangev[2]-rangev[1])/
+   (rangev[4]-rangev[3])*
+   heightp/widthp)/(0.25*pi))
```

Performance of Efficient Portfolio for Stocks and Bonds

```

> # Calculate cumulative returns of VTI and IEF
> retsoptim <- lapply(retp, function(retp) exp(cumsum(retp)))
> retsoptim <- rutils::do_call(cbind, retsoptim)
> # Calculate the efficient portfolio returns
> retsoptim <- cbind(exp(cumsum(retp %*%
+   c(weightv[whichmax], 1-weightv[whichmax]))),
+   retsoptim)
> colnames(retsoptim)[1] <- "efficient"
> # Plot efficient portfolio with custom line colors
> themev <- chart_theme()
> themev$col$line.col <- c("orange", "blue", "green")
> chart_Series(retsoptim, theme=themev,
+   name="Efficient Portfolio for Stocks and Bonds")
> legend("top", legend=colnames(retsoptim),
+   cex=0.8, inset=0.1, bg="white", lty=1,
+   lwd=6, col=themev$col$line.col, bty="n")

```



draft: Efficient Frontier of Principal Components

First apply Principal Component Analysis and then perform portfolio optimization using the principal components.

If the correlations of returns are equal to zero, then the covariance matrix is diagonal:

$$\mathbb{C} = \begin{pmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^2 \end{pmatrix}$$

Where σ_i^2 is the variance of returns of asset i .

The inverse of $\mathbb{C}$ is then simply:

$$\mathbb{C}^{-1} = \begin{pmatrix} \sigma_1^{-2} & 0 & \cdots & 0 \\ 0 & \sigma_2^{-2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^{-2} \end{pmatrix}$$

The *minimum variance* portfolio weights are proportional to the inverse of the individual variances:

$$w_i = \frac{1}{\sigma_i^2 \sum_{i=1}^n \sigma_i^{-2}}$$

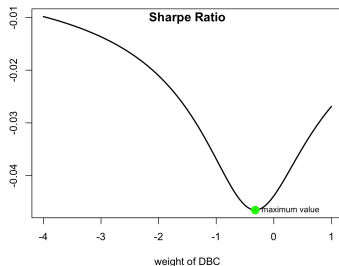
The *maximum Sharpe* portfolio weights are proportional to the ratio of excess returns divided by the individual variances:

$$w_i = \frac{\mu_i}{\sigma_i^2 \sum_{i=1}^n \mu_i \sigma_i^{-2}}$$

Sharpe Ratio Objective Function

The function `optimize()` performs *one-dimensional* optimization over a single independent variable, in the specified interval.

```
> # Calculate daily ETF percentage returns
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> nrows <- NROW(retp)
> # Create initial vector of portfolio weights
> weightv <- rep(1, NROW(symbolv))
> names(weightv) <- symbolv
> # Objective equal to minus Sharpe ratio
> objfun <- function(weightv, retp) {
+   retportf <- retp %*% weightv
+   stdev <- sd(retportf)
+   if (stdev == 0)
+     return(0)
+   else
+     return(-mean(retportf)/stdev)
+ } ## end objfun
> # Objective for equal weight portfolio
> objfun(weightv, retp=retp)
> optiml <- unlist(optimize(f=function(weightv)
+   objfun(c(1, 1, weightv), retp=retp),
+   interval=c(-10, 10)))
> # Vectorize objective function with respect to third weight
> objvec <- function(weightv) sapply(weightv,
+   function(weightv) objfun(c(1, 1, weightv), retp=retp))
> # Or
> objvec <- Vectorize(FUN=function(weightv)
+   objfun(c(1, 1, weightv), retp=retp),
+   vectorize.args="weightv") ## end Vectorize
> objvec(1)
> objvec(1:3)
```



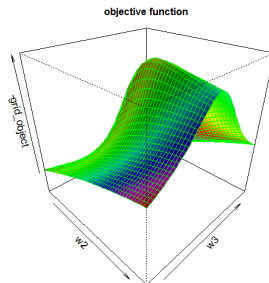
```
> # Plot objective function with respect to third weight
> curve(expr=objvec, type="l", xlim=c(-4.0, 1.0),
+   xlab=paste("weight of", names(weightv[3])),
+   ylab="", lwd=2)
> title(main="Sharpe Ratio", line=(-1)) ## Add title
> points(x=optiml[1], y=optiml[2], col="green", lwd=6)
> text(x=optiml[1], y=optiml[2],
+   labels="maximum value", pos=4, cex=0.8)
>
> ### Below is simplified plotting of objective function
> # Create vector of DBC weights
> weightv <- seq(from=-4, to=1, by=0.1)
> objv <- sapply(weightv, function(weightv)
+   objfun(c(1, 1, weightv), retp))
> plot(x=weightv, y=objv, t="l",
+   xlab="weight of DBC", ylab="", lwd=2)
> title(main="Sharpe Ratio", line=(-1)) ## Add title
> points(x=optiml[1], y=optiml[2], col="green", lwd=6)
> text(x=optiml[1], y=optiml[2],
```

Perspective Plot of Portfolio Objective Function

The function `persp()` plots a 3d perspective surface plot of a function specified over a grid of argument values.

The function `outer()` calculates the values of a function over a grid spanned by two variables, and returns a matrix of function values.

The package *rgl* allows creating *interactive* 3d scatterplots and surface plots including perspective plots, based on the *OpenGL* framework.



```
> # Vectorize function with respect to two weights
> objvec <- Vectorize(
+   FUN=function(w1, w2, w3) objfun(c(w1, w2, w3), ret),
+   vectorize.args=c("w2", "w3")) ## end Vectorize
> # Calculate objective on 2-d (w2 x w3) parameter grid
> w2 <- seq(-3, 7, length=50)
> w3 <- seq(-5, 5, length=50)
> gridm <- outer(w2, w3, FUN=objvec, w1=1)
> rownames(gridm) <- round(w2, 2)
> colnames(gridm) <- round(w3, 2)
> # Perspective plot of objective function
> persp(w2, w3, -gridm,
+   theta=45, phi=30, shade=0.5,
+   col="rainbow(50)", border="green",
+   main="objective function")
```

```
> # Interactive perspective plot of objective function
> library(rgl)
> rgl::persp3d(z=-gridm, zlab="objective",
+   col="green", main="objective function")
> rgl::persp3d(
+   x=function(w2, w3) {objvec(w1=1, w2, w3)},
+   xlim=c(-3, 7), ylim=c(-5, 5),
+   col="green", axes=FALSE)
> # Render the 3d surface plot of function
> rgl::rglwidget(elementId="plot3drgl", width=1000, height=1000)
```

Portfolio Optimization Using optim()

The functional `optim()` performs *multi-dimensional* optimization.

The argument `par` are the initial parameter values.

The argument `fn` is the objective function to be minimized.

The argument of the objective function which is to be optimized, must be a vector argument.

`optim()` accepts additional parameters bound to the dots `"..."` argument, and passes them to the `fn` objective function.

The arguments `lower` and `upper` specify the search range for the variables of the objective function `fn`.

`method="L-BFGS-B"` specifies the quasi-Newton optimization method.

The parameter `control=list(factr=1e5)` determines the precision of the L-BFGS-B method. A smaller `factr` value produces results with a higher precision, but it requires more iterations and a longer optimization. A larger `factr` value achieves a faster solution but less precise results.

`optim()` returns a list containing the location of the minimum and the objective function value.

```
> # Create initial vector of portfolio weights
> weightv <- rep(1, NROW(symbolv))
> names(weightv) <- symbolv
> # Optimization to find weights with maximum Sharpe ratio
> optiml <- optim(par=weightv,
+               fn=objfun,
+               retp=retp,
+               method="L-BFGS-B",
+               control=list(factr=1e5),
+               upper=c(10, 10, 10),
+               lower=c(-10, -10, -10))
> # Optimal parameters
> weightv <- optiml$par
> weightv <- weightv/sqrt(sum(weightv^2))
> weightv
> # Optimal Sharpe ratio
> -objfun(weightv, retp)
> # Calculate the weights from the inverse covariance matrix
> weightv <- drop(solve(cov(retp)) %*% sapply(retp, mean))
> weightv <- weightv/sqrt(sum(weightv^2))
> weightv
```

Optimization Algorithms Using `optim()`

Table: Comparison of optimization methods for portfolio optimization

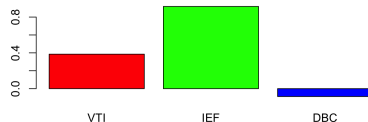
Method	Type	Strengths	Weaknesses	Suitability for Portfolio Optimization
BFGS	Quasi-Newton	Fast convergence, uses gradient + Hessian approximation	Memory-heavy for large problems	Strong choice for medium-sized portfolios
L-BFGS-B	Limited-memory BFGS with box constraints	Handles bounds (e.g., weights ≥ 0), memory-efficient	Requires smooth objective	Best overall for portfolio optimization with constraints
CG (Conjugate Gradient)	Gradient-based	Efficient for large-scale problems	Slower convergence than BFGS	Good for very large portfolios
Nelder-Mead	Derivative-free simplex	Works without gradients, robust for small problems	Slow, scales poorly in high dimensions	Limited use; not ideal for large portfolios
Brent	1D derivative-free	Very efficient in one dimension	Only works for single-variable problems	Not applicable to portfolios
SANN (Simulated Annealing)	Stochastic global search	Escapes local minima, works on non-smooth functions	Very slow, no guarantee of optimality	Useful for highly non-convex problems

Optimized Portfolio Returns

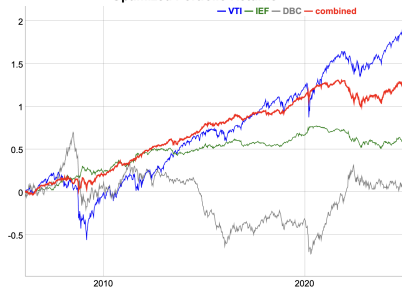
The optimized portfolio has both long and short positions, and has a larger *Sharpe ratio* than the individual assets.

```
> # barplot of optimal portfolio weights
> barplot(weightv, col=c("red", "green", "blue"),
+   main="Optimized portfolio weights")
> # Calculate the cumulative wealth of the optimized portfolio
> wealthv <- cbind(retp %*% weightv, retp)
> colnames(wealthv)[1] <- "combined"
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   (mean(x)-raterf)/c(Sharpe=sd(x), Sortino=sd(x[x<0]))))
> # Plot the log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> colv <- colnames(wealthv)
> colr <- c("red", "blue", "green", "grey")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimized Portfolio Returns") %>%
+   dyOptions(colors=colr, strokeWidth=1) %>%
+   dySeries(name=colv[1], col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Optimized portfolio weights



Optimized Portfolio Returns



draft: Mean-Variance Portfolio Optimization

The mean-variance objective function is designed to maximize portfolio returns and minimize their variance:

$$O(x) = \mathbf{w}^T \mathbb{C} \mathbf{w} - q \mathbf{w}^T \mathbf{r}$$

Where $\mathbb{C}$ is the covariance matrix of returns, $\mathbf{r}$ is the vector of returns, $\mathbf{w}$ is the vector of portfolio weights, and q is the risk tolerance factor.

The mean-variance optimal portfolio is defined as

$$\theta_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta | x) \mathbf{w}^T \mathbb{C} \mathbf{w}$$

Where the sum of portfolio weights $\mathbf{w}$ is constrained to equal 1: $\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$.

Legacy stuff below:

A Linear Regression model with p explanatory variables $\{x_j\}$, is defined by the formula:

$$z_i = \alpha + \sum_{j=1}^k \beta_j x_{i,j} + \varepsilon_i$$

Or in vector notation:

$$z = \alpha + \beta x + \varepsilon$$

The response variable z and the p explanatory variables $\{x_j\}$ each contain n observations.

The response variable z is a vector of length n , and the explanatory variable x is a (n, p) -dimensional matrix.

The OLS estimate for α is given by:

$$\alpha = z^T \mathbf{1} - \beta x^T \mathbf{1}$$

If the variables are centered, then the OLS estimate for β is given by equating the RSS derivative to zero:

$$RSS_{\beta} = -2(z - \beta x)^T x = 0$$

$$x^T z - \beta x^T x = 0$$

$$\beta = (x^T x)^{-1} x^T z$$

The matrix $x^T x$ is the covariance matrix of the matrix x .

The covariance matrix $x^T x$ is invertible if the columns of x are linearly independent.

The matrix $(x^T x)^{-1} x^T$ is known as the *Moore-Penrose pseudo-inverse* of the matrix x .

In the special case when the inverse matrix x^{-1} does exist, then the *pseudo-inverse* matrix simplifies to the inverse: $(x^T x)^{-1} x^T = x^{-1} (x^T)^{-1} x^T = x^{-1}$

Package *DEoptim* for Global Optimization

The function `DEoptim()` from package *DEoptim* performs *global* optimization using the *Differential Evolution* algorithm.

Differential Evolution is a genetic algorithm which evolves a population of solutions over several generations,

<https://link.springer.com/content/pdf/10.1023/A:1008202821328.pdf>

The first generation of solutions is selected randomly.

Each new generation is obtained by combining solutions from the previous generation.

The best solutions are selected for creating the next generation.

The *Differential Evolution* algorithm is well suited for very large multi-dimensional optimization problems, such as portfolio optimization.

Gradient optimization methods are more efficient than *Differential Evolution* for smooth objective functions with no local minima.

```
> # Rastrigin function with vector argument for optimization
> rastrigin <- function(vecv, param=25){
+   sum(vecv^2 - param*cos(vecv))
+ } ## end rastrigin
> vecv <- c(pi/6, pi/6)
> rastrigin(vecv=vecv)
> library(DEoptim)
> # Optimize rastrigin using DEoptim
> optim1 <- DEoptim::DEoptim(rastrigin,
+   upper=c(6, 6), lower=c(-6, -6),
+   DEoptim.control(trace=FALSE, itermax=50))
> # Optimal parameters and value
> optim1$optim$bestmem
> rastrigin(optim1$optim$bestmem)
> summary(optim1)
> plot(optim1)
```

Portfolio Optimization Using Package *DEoptim*

The *Differential Evolution* algorithm is well suited for very large multi-dimensional optimization problems, such as portfolio optimization.

With the parameter `control=list(parallelType=1)`, `DEoptim::DEoptim()` performs the optimization using parallel computing.

With the parameter `control=list(itermax=100)`, `DEoptim::DEoptim()` performs at most 100 iterations.

To achieve a good precision, `itermax` should be set to a much larger value, such as 10000. 100 iterations.

```
> # Perform optimization using DEoptim
> optim1 <- DEoptim::DEoptim(fn=objfun,
+   upper=rep(10, NCOL(retp)),
+   lower=rep(-10, NCOL(retp)),
+   retp=retp,
+   control=list(trace=FALSE, itermax=100, parallelType=1))
> weightv <- optim1$optim$bestmem
> names(weightv) <- colnames(retp)
> weightv <- weightv/sqrt(sum(weightv^2))
> weightv
```

Efficient Portfolios With Ridge Regularization

Portfolio optimization has a very large number of parameters, equal to the number of stocks in the portfolio. This causes the *overfitting* of the stock portfolio weights to the in-sample returns.

The technique of *regularization* is designed to reduce the overfitting of the parameters in a model.

Ridge regularization adds a penalty term to the objective function, proportional to the sum of squares of the weights: $\mathbf{w}^T \mathbf{w} = \sum_{i=1}^n w_i^2$.

The Lagrangian for the efficient portfolio weights with *ridge regularization* is:

$$\mathcal{L} = \mathbf{w}^T \mathbb{C} \mathbf{w} - \lambda_1 (\mathbf{w}^T \mathbf{1} + 1) - \lambda_2 (\mathbf{w}^T \mathbf{r} - r_t) + \lambda \mathbf{w}^T \mathbf{w}$$

Where the parameter λ determines the amount of regularization.

The Lagrangian can be rewritten using the ridge covariance matrix $\mathbb{C}_r = \mathbb{C} + \lambda \mathbf{1}$:

$$\mathcal{L} = \mathbf{w}^T \mathbb{C}_r \mathbf{w} - \lambda_1 (\mathbf{w}^T \mathbf{1} - 1) - \lambda_2 (\mathbf{w}^T \mathbf{r} - r_t)$$

This is the efficient portfolio Lagrangian with the ridge covariance matrix $\mathbb{C}_r$.

Maximum Sharpe Portfolio With Ridge Regularization

Maximizing the efficient portfolio Lagrangian is equivalent to maximizing the *Sharpe ratio*:

$$SR = \frac{\mathbf{w}^T \boldsymbol{\mu}}{\sigma}$$

Where $\boldsymbol{\mu} = \mathbf{r} - r_f$ is the vector of excess returns, and $\sigma = \sqrt{\mathbf{w}^T \mathbb{C}_r \mathbf{w}}$ is the standard deviation of portfolio with the ridge covariance matrix $\mathbb{C}_r = \mathbb{C} + \lambda \mathbf{1}$.

The portfolio weights satisfying the linear constraint $\mathbf{w}^T \mathbf{1} = 1$ are:

$$\mathbf{w} = \frac{\mathbb{C}_r^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbb{C}_r^{-1} \boldsymbol{\mu}}$$

These are the weights of the *maximum Sharpe* portfolio, with the vector of mean excess returns equal to $\boldsymbol{\mu}$, and the covariance matrix equal to $\mathbb{C}_r$.

```
> # Objective equal to minus ridge Sharpe ratio
> objfun <- function(weightv, retx, lambdaf=0) {
+   retportf <- retx %*% weightv
+   stdev <- sd(retportf)
+   if (stdev == 0)
+     return(0)
+   else
+     return(-mean(retportf)/stdev + lambdaf*sum(weightv^2))
+ } ## end objfun
> # Create initial vector of portfolio weights
> weightv <- rep(1, NROW(symbolv))
> names(weightv) <- symbolv
> # Calculate vector of excess returns
> raterf <- 0.03/252
> retx <- (retp - raterf)
> # Calculate the weights using optimization
> lambdaf <- 300.0
> objfun(weightv, retp=retx, lambdaf=lambdaf)
> optim1 <- optim(par=weightv,
+   fn=objfun,
+   retx=retx,
+   lambdaf=lambdaf,
+   method="L-BFGS-B",
+   upper=c(10, 10, 10),
+   lower=c(-10, -10, -10))
> weightv <- optim1$par
> names(weightv) <- symbolv
> # Calculate ridge covariance matrix of returns and its inverse
> covmat <- cov(retp) + lambdaf*diag(ncol(retp))
> covinv <- solve(a=covmat)
> # Calculate the maximum Sharpe weights
> retm <- colMeans(retx)
> weightms <- drop(covinv %*% retm)/sum(covinv %*% retm)
> all.equal(weightv, weightms)
```

Portfolio Optimization With *Elastic Net Regularization*

The technique of *regularization* is designed to reduce the number of parameters in a model, for example in portfolio optimization.

The *regularization* techniques often add a penalty term to the objective function.

The penalty term in *ridge* regularization is the sum of squares of the weights: $\sum_{i=1}^n w_i^2$. The *ridge* penalty shrinks (reduces) the weights together, without shrinking some of them to zero.

The penalty term in *LASSO* regularization is the sum of the absolute values of the weights: $\sum_{i=1}^n |w_i|$. The *LASSO* penalty shrinks the less important weights to zero, similar to dimension reduction.

The *elastic net* regularization is a combination of *ridge* regularization and *LASSO* regularization:

$$w_{max} = \arg \max_w \left[\frac{\mathbf{w}^T \boldsymbol{\mu}}{\sigma} - \lambda((1-\alpha) \sum_{i=1}^n w_i^2 + \alpha \sum_{i=1}^n |w_i|) \right]$$

The λ parameter determines the amount of regularization, and α controls the balance between the *ridge* and *LASSO* penalties.

```
> # Objective with regularization penalty
> objfun <- function(weightv, retp, lambdaf, alphaf) {
+   retportf <- retp %*% weightv
+   stdev <- sd(retportf)
+   if (stdev == 0)
+     return(0)
+   else {
+     penaltyv <- lambdaf*((1-alphaf)*sum(weightv^2) +
+     alphaf*sum(abs(weightv)))
+     return(-mean(retportf)/stdev + penaltyv)
+   } ## end if
+ } ## end objfun
> # Objective for equal weight portfolio
> weightv <- rep(1, NROW(symbolv))
> names(weightv) <- symbolv
> lambdaf <- 0.5 ; alphaf <- 0.5
> objfun(weightv, retp=retp, lambdaf=lambdaf, alphaf=alphaf)
> # Perform optimization using DEoptim
> optim1 <- DEoptim::DEoptim(fn=objfun,
+   upper=rep(10, NCOL(retp)),
+   lower=rep(-10, NCOL(retp)),
+   retp=retp,
+   lambdaf=lambdaf,
+   alphaf=alphaf,
+   control=list(trace=FALSE, itermax=100, parallelType=1))
> weightv <- optim1$optim$bestmem
> names(weightv) <- colnames(retp)
> weightv <- weightv/sqrt(sum(weightv^2))
> weightv
```

Optimal Stock Portfolio Weights

The portfolio weights obtained from inverting the covariance matrix are the same as the weights obtained from portfolio optimization.

```
> # Load stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RDa")
> datev <- zoo::index(na.omit(retstock$GOOGL))
> retp <- retstock[datev] ## Subset the returns to GOOGL
> # Remove the stocks with any NA values
> numna <- sapply(retp, function(x) sum(is.na(x)))
> retp <- retp[, numna==0]
> # Select 100 random stocks
> retp <- retp[, sample(NCOL(retp), 100)]
> symbolv <- colnames(retp)
> datev <- zoo::index(retp)
> retis <- retp["/2014"] ## In-sample returns
> raterf <- 0.03/252
> retx <- (retis - raterf) ## Excess returns
> # Calculate the maximum Sharpe weights in-sample interval
> colmeanv <- colMeans(retx, na.rm=TRUE)
> covmat <- cov(retx, use="pairwise.complete.obs")
> invreg <- MASS::ginv(covmat)
> weightv <- drop(invreg %*% colmeanv)
> names(weightv) <- symbolv
```

```
> # Calculate the weights using optimization without regularization
> lambdaf <- 0.0 ; alphaf <- 1.0
> optim1 <- optim(par=weightv,
+               fn=objfun,
+               retp=retp,
+               lambdaf=lambdaf,
+               alphaf=alphaf,
+               method="L-BFGS-B",
+               control=list(factr=1e5),
+               upper=c(10, 10, 10),
+               lower=c(-10, -10, -10))
> weighto <- optim1$par
> names(weighto) <- symbolv
> all.equal(weighto, weightv)
```

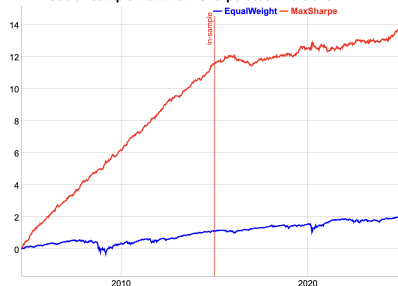
draft: Portfolio Weights With Regularization

The *portfolio optimization* strategy for stocks is *overfit* in the *in-sample* interval.

Therefore the strategy performance is poor in the *out-of-sample* interval.

```
> # Calculate the portfolio weights with regularization
> lambdav <- seq(1.905e-7, 1.90525e-7, length=5)
> lambdav <- seq(0, 1e-6, length=7)
> ncores <- detectCores() - 1
> weightm <- mclapply(lambdav, function(lambdaf) {
+   optiml <- optim(par=weightv, fn=objfun,
+     retp=retx,
+     lambdaf=lambdaf, alphaf=0,
+     method="L-BFGS-B",
+     control=list(factr=1e5),
+     upper=c(10, 10, 10),
+     lower=c(-10, -10, -10))
+   optiml$par
+ }) ## end mclapply
> weightm <- do.call(cbind, weightm)
> colnames(weightm) <- paste0("lambda=", lambdav)
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(weightm))
> plot(sort(weightm[, 1]), xlab="NA", ylab="weight", col=colorv[1],
+   for (i in 2:NCOL(weightm)) {
+     points(sort(weightm[, i]), col=colorv[i], lwd=2)
+   } ## end for
> apply(weightm, 2, function(x) sum(abs(x)))
>
> lambdav <- seq(0, 1e-6, length=7)
> ncores <- detectCores() - 1
> weightm <- mclapply(lambdav, function(lambdaf) {
+   optiml <- DEoptim(fn=objfun,
+     upper=rep(10, NCOL(retp)),
+     lower=rep(-10, NCOL(retp)),
+     retp=retx,
```

Out-of-Sample Maximum Sharpe Stock Portfolio



```
> wealthv <- cbind(pnlmaxs, pnls)
> colnames(wealthv) <- c("MaxSharpe", "Shrink")
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014/", function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["2015/", function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of cumulative portfolio returns
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Out-of-Sample Maximum Sharpe Stock Portfolio") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(end(retis[, 1]), label="in-sample", strokePattern="solid")
+   dyLegend(width=300)
```

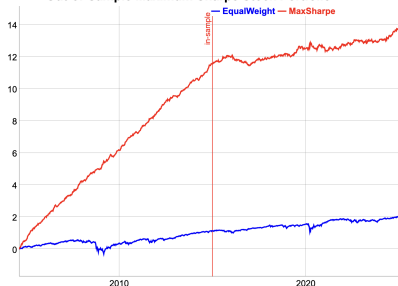

Optimal Stock Portfolio Out-of-Sample

The *portfolio optimization* strategy for stocks is *overfit* in the *in-sample* interval.

Therefore the strategy performance is poor in the *out-of-sample* interval.

```
> # Load S&P 500 stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RData")
> datev <- zoo::index(na.omit(retstock$GOOGL))
> retp <- retstock[datev] ## Subset the returns to GOOGL
> # Remove the stocks with any NA values
> numna <- sapply(retp, function(x) sum(is.na(x)))
> retp <- retp[, numna==0]
> # Select 100 random stocks
> retp <- retp[, sample(NCOL(retp), 100)]
> symbolv <- colnames(retp)
> datev <- zoo::index(retp)
> retis <- retp["/2014"] ## In-sample returns
> raterf <- 0.03/252
> retx <- (retis - raterf) ## Excess returns
> # Maximum Sharpe weights in-sample interval
> colmeanv <- colMeans(retx, na.rm=TRUE)
> covmat <- cov(retx, use="pairwise.complete.obs")
> invreg <- MASS::ginv(covmat)
> weightv <- drop(invreg %*% colmeanv)
> names(weightv) <- symbolv
> # Calculate the maximum Sharpe portfolio returns
> pnlmaxs <- HighFreq::mult_mat(weightv, retp)
> pnlmaxs <- rowMeans(pnlmaxs, na.rm=TRUE)
> pnlmaxs <- xts::xts(pnlmaxs, datev)
> retew <- xts::xts(rowMeans(retp, na.rm=TRUE), datev)
> pnlmaxs <- pnlmaxs*sd(retew["/2014"])/sd(pnlmaxs["/2014"])
> # Calculate the regularization portfolio returns
> lambdaf <- 1.0 ; alphaf <- 1.0
> optiml <- optim(par=weightv,
+               fn=objfun,
+               retp=retx,
```

Out-of-Sample Maximum Sharpe Stock Portfolio



```
> wealthv <- cbind(pnlmaxs, pnls)
> colnames(wealthv) <- c("MaxSharpe", "Shrink")
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["2015/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of cumulative portfolio returns
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Out-of-Sample Maximum Sharpe Stock Portfolio") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(end(retis[1]), label="in-sample", strokePattern="solid")
+   dyLegend(width=300)
```

Optimal Portfolios Under Zero Correlation

If the correlations of returns are equal to zero, then the covariance matrix is diagonal:

$$\mathbb{C} = \begin{pmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^2 \end{pmatrix}$$

Where σ_i^2 is the variance of returns of asset i .

The inverse of $\mathbb{C}$ is then simply:

$$\mathbb{C}^{-1} = \begin{pmatrix} \sigma_1^{-2} & 0 & \cdots & 0 \\ 0 & \sigma_2^{-2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^{-2} \end{pmatrix}$$

The *minimum variance* portfolio weights are proportional to the inverse of the individual variances:

$$w_i = \frac{1}{\sigma_i^2 \sum_{i=1}^n \sigma_i^{-2}}$$

The *maximum Sharpe* portfolio weights are proportional to the ratio of the excess returns divided by the individual variances:

$$w_i = \frac{\mu_i}{\sigma_i^2 \sum_{i=1}^n \mu_i \sigma_i^{-2}}$$

The portfolio weights are proportional to the *Kelly ratios* - the excess returns divided by the variances:

$$w_i \propto \frac{\mu_i}{\sigma_i^2}$$

draft: Portfolio Optimization Using Principal Components

First apply Principal Component Analysis and then perform portfolio optimization using the principal components.

If the correlations of returns are equal to zero, then the covariance matrix is diagonal:

$$\mathbb{C} = \begin{pmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^2 \end{pmatrix}$$

Where σ_i^2 is the variance of returns of asset i .

The inverse of $\mathbb{C}$ is then simply:

$$\mathbb{C}^{-1} = \begin{pmatrix} \sigma_1^{-2} & 0 & \cdots & 0 \\ 0 & \sigma_2^{-2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^{-2} \end{pmatrix}$$

The *minimum variance* portfolio weights are proportional to the inverse of the individual variances:

$$w_i = \frac{1}{\sigma_i^2 \sum_{i=1}^n \sigma_i^{-2}}$$

The *maximum Sharpe* portfolio weights are proportional to the ratio of excess returns divided by the individual variances:

$$w_i = \frac{\mu_i}{\sigma_i^2 \sum_{i=1}^n \mu_i \sigma_i^{-2}}$$

draft: Portfolio Optimization Packages in R

The following R packages provide functions for portfolio optimization:

- package *PortfolioAnalytics*: relies on packages *xts*, *ROI*, and *DEoptim*,
- package *parma*: relies on packages *xts*, *Rglpk*, and *quadprog*,
- package *fPortfolio* from the *Rmetrics* suite: relies on packages *tseries*, *Rglpk*, and *quadprog*,

These portfolio optimization packages call generic optimization functions written in compiled C++

```
> # Portfolio optimization
```

Maximum Return Portfolio Using Linear Programming

The stock weights of the maximum return portfolio are obtained by maximizing the portfolio returns:

$$w_{max} = \arg \max_w [\bar{\mathbf{r}}^T \mathbf{w}] = \arg \max_w \left[\sum_{i=1}^n w_i r_i \right]$$

Where $\mathbf{r}$ is the vector of returns, and $\mathbf{w}$ is the vector of portfolio weights, with a linear constraint:

$$\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$$

And a box constraint:

$$0 \leq w_i \leq 1$$

The weights of the maximum return portfolio can be calculated using linear programming (LP), which is the optimization of linear objective functions subject to linear constraints.

The function `Rglpk_solve_LP()` from package *Rglpk* solves linear programming problems by calling the *GNU Linear Programming Kit* library.

```
> library(rutils)
> library(Rglpk)
> # Vector of symbol names
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> # Calculate the objective vector - the mean returns
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> objvec <- colMeans(retp)
> # Specify matrix of linear constraint coefficients
> coeffm <- matrix(c(rep(1, nstocks), 1, 1, 0),
+                 nc=nstocks, byrow=TRUE)
> # Specify the logical constraint operators
> logop <- c("=", "<=")
> # Specify the vector of constraints
> consv <- c(1, 0)
> # Specify box constraints (-1, 1) (default is c(0, Inf))
> boxc <- list(lower=list(ind=1:nstocks, val=rep(-1, nstocks)),
+             upper=list(ind=1:nstocks, val=rep(1, nstocks)))
> # Perform optimization
> optiml <- Rglpk::Rglpk_solve_LP(
+   obj=objvec,
+   mat=coeffm,
+   dir=logop,
+   rhs=consv,
+   bounds=boxc,
+   max=TRUE)
> all.equal(optiml$optimum, sum(objvec*optiml$solution))
> optiml$solution
> coeffm %*% optiml$solution
```

Conditional Value at Risk (CVaR)

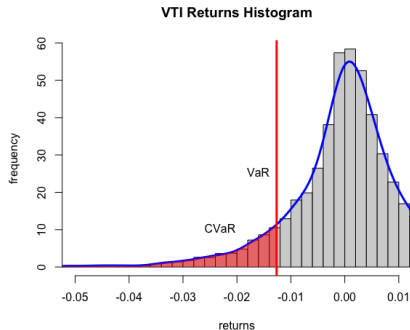
The *Conditional Value at Risk (CVaR)* is equal to the average of the *VaR* for confidence levels less than a given confidence level α :

$$\text{CVaR} = \frac{1}{\alpha} \int_0^\alpha \text{VaR}(p) dp$$

The *Conditional Value at Risk* is also called the *Expected Shortfall (ES)*, or the *Expected Tail Loss (ETL)*.

The function `density()` calculates a kernel estimate of the probability density for a sample of data, and returns a list with a vector of loss values and a vector of corresponding densities.

```
> # Calculate the VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> confl <- 0.1
> varisk <- quantile(retp, confl)
> cvar <- mean(retp[retp < varisk])
> # Or
> sortv <- sort(as.numeric(retp))
> varind <- round(confl*NROW(retp))
> varisk <- sortv[varind]
> cvar <- mean(sortv[1:varind])
> # Plot histogram of VTI returns
> varmin <- (-0.05)
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=100, xlim=c(varmin, 0.01),
+   ylab="frequency", freq=FALSE, main="VTI Returns Histogram")
```



```
> # Plot density of losses
> densv <- density(retp, adjust=1.5)
> lines(densv, lwd=3, col="blue")
> # Add line for VaR
> abline(v=varisk, col="red", lwd=3)
> ymax <- max(densv$y)
> text(x=varisk, y=2*ymax/3, labels="VaR", lwd=2, pos=2)
> # Add shading for CVaR
> rangev <- (densv$x < varisk) & (densv$x > varmin)
> polygon(
+   c(varmin, densv$x[rangev], varisk),
+   c(0, densv$y[rangev], 0),
+   col=rgb(1, 0, 0, 0.5), border=NA)
> text(x=1.5*varisk, y=ymax/7, labels="CVaR", lwd=2, pos=2)
```

CVaR Portfolio Weights Using Linear Programming

The stock weights of the minimum *CVaR* portfolio can be calculated using linear programming (*LP*), which is the optimization of linear objective functions subject to linear constraints,

$$w_{min} = \arg \max_{\mathbf{w}} \left[\sum_{i=1}^n w_i b_i \right]$$

Where b_i is the negative objective vector, and $\mathbf{w}$ is the vector of portfolio weights, with a linear constraint:

$$\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$$

And a box constraint:

$$0 \leq w_i \leq 1$$

The function `Rglpk_solve_LP()` from package *Rglpk* solves linear programming problems by calling the *GNU Linear Programming Kit* library.

```
> library(rutils) ## Load rutils
> library(Rglpk)
> # Vector of symbol names and returns
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> retp <- colMeans(rutils::etfenv$returns[, symbolv])
> retm <- colMeans(retp)
> confl <- 0.05
> rmin <- 0 ; wmin <- 0 ; wmax <- 1
> weightsum <- 1
> ncols <- NCOL(retp) ## number of assets
> nrows <- NROW(retp) ## number of rows
> # Create objective vector
> objvec <- c(numeric(ncols), rep(-1/(confl/nrows), nrows), -1)
> # Specify matrix of linear constraint coefficients
> coeff <- rbind(cbind(rbind(1, retm),
+                       matrix(data=0, nrow=2, ncol=(nrows+1))),
+               cbind(coredata(retp), diag(nrows), 1))
> # Specify the logical constraint operators
> logop <- c("=", ">=", rep(">=", nrows))
> # Specify the vector of constraints
> consv <- c(weightsum, rmin, rep(0, nrows))
> # Specify box constraints (wmin, wmax) (default is c(0, Inf))
> boxc <- list(lower=list(ind=1:ncols, val=rep(wmin, ncols)),
+             upper=list(ind=1:ncols, val=rep(wmax, ncols)))
> # Perform optimization
> optiml <- Rglpk_solve_LP(obj=objvec, mat=coeff, dir=logop, rhs=consv,
+                          box=boxc, all.equal=TRUE, sum=objvec*optiml$solution)
> all.equal(optiml$optimum, sum(objvec*optiml$solution))
> coeff %*% optiml$solution
> as.numeric(optiml$solution[1:ncols])
```

Package *quadprog* for Quadratic Programming

Quadratic programming (*QP*) is the optimization of quadratic objective functions subject to linear constraints.

Let $O(x)$ be an objective function that is quadratic with respect to a vector variable x :

$$O(x) = \frac{1}{2}x^T Qx - d^T x$$

Where Q is a *positive definite* matrix ($x^T Qx > 0$), and d is a vector.

An example of a *positive definite* matrix is the covariance matrix of linearly independent variables.

Let the linear constraints on the variable x be specified as:

$$Ax \geq b$$

Where A is a matrix, and b is a vector.

The function `solve.QP()` from package *quadprog* performs optimization of quadratic objective functions subject to linear constraints.

```
> library(quadprog)
> # Minimum variance weights without constraints
> optim1 <- solve.QP(Dmat=2*covmat,
+   dvec=rep(0, 2),
+   Amat=matrix(0, nr=2, nc=1),
+   bvec=0)
> # Minimum variance weights sum equal to 1
> optim1 <- solve.QP(Dmat=2*covmat,
+   dvec=rep(0, 2),
+   Amat=matrix(1, nr=2, nc=1),
+   bvec=1)
> # Optimal value of objective function
> t(optim1$solution) %*% covmat %*% optim1$solution
> ## Perform simple optimization for reference
> # Objective function for simple optimization
> objfun <- function(x) {
+   x <- c(x, 1-x)
+   t(x) %*% covmat %*% x
+ } ## end objfun
> unlist(optimize(f=objfun, interval=c(-1, 2)))
```


Portfolio Optimization Using Package *quadprog*

The objective function is designed to minimize portfolio variance and maximize its returns:

$$O(x) = \mathbf{w}^T \mathbb{C} \mathbf{w} - \mathbf{w}^T \mathbf{r}$$

Where $\mathbb{C}$ is the covariance matrix of returns, $\mathbf{r}$ is the vector of returns, and $\mathbf{w}$ is the vector of portfolio weights.

The portfolio weights $\mathbf{w}$ are constrained as:

$$\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$$

$$0 \leq w_i \leq 1$$

The function `solve.QP()` has the arguments:

`Dmat` and `dvec` are the matrix and vector defining the quadratic objective function.

`Amat` and `bvec` are the matrix and vector defining the constraints.

`meq` specifies the number of equality constraints (the first `meq` constraints are equalities, and the rest are inequalities).

```
> # Calculate daily ETF percentage returns
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> # Calculate the covariance matrix
> covmat <- cov(retp)
> # Minimum variance weights, with sum equal to 1
> optim1 <- quadprog::solve.QP(Dmat=2*covmat,
+                               dvec=numeric(3),
+                               Amat=matrix(1, nr=3, nc=1),
+                               bvec=1)
> # Minimum variance, maximum returns
> optim1 <- quadprog::solve.QP(Dmat=2*covmat,
+                               dvec=apply(0.1*retp, 2, mean),
+                               Amat=matrix(1, nr=3, nc=1),
+                               bvec=1)
> # Minimum variance positive weights, sum equal to 1
> a_mat <- cbind(matrix(1, nr=3, nc=1),
+                  diag(3), -diag(3))
> b_vec <- c(1, rep(0, 3), rep(-1, 3))
> optim1 <- quadprog::solve.QP(Dmat=2*covmat,
+                               dvec=numeric(3),
+                               Amat=a_mat,
+                               bvec=b_vec,
+                               meq=1)
```

Package *PortfolioAnalytics*

The package *PortfolioAnalytics* contains functions and data sets for portfolio optimization.

The function `data()` loads external data or lists data sets in a package.

```
> library(PortfolioAnalytics) ## load package "PortfolioAnalytics"
> # get documentation for package "PortfolioAnalytics"
> packageDescription("PortfolioAnalytics") ## get short description
>
> help(package="PortfolioAnalytics") ## load help page
>
> data(package="PortfolioAnalytics") ## list all datasets in "PortfolioAnalytics"
>
> ls("package:PortfolioAnalytics") ## list all objects in "PortfolioAnalytics"
>
> detach("package:PortfolioAnalytics") ## remove PortfolioAnalytics from search
```

Portfolio Definition

Portfolios are defined by a named vector of asset weights, and portfolio constraints and objectives.

`portfolio.spec` creates a portfolio object that contains asset weights, constraints, and objectives.

`add.constraint` adds or updates constraints on of the portfolio object.

`add.objective` adds or updates risk/return objectives of the portfolio object.

```
> library(PortfolioAnalytics)
> # Use ETF returns from package rutils
> library(rutils)
> symbolv <- c("VTI", "IEF", "DBC", "XLF",
+ "VNQ", "XLP", "XLV", "XLU", "XLB", "XLE")
> # Initial portfolio to equal weights
> portfi <- rep(1/NROW(symbolv), NROW(symbolv))
> # named vector
> names(portfi) <- symbolv
> # Create portfolio object
> portfi <- portfolio.spec(assets=portfi)
```

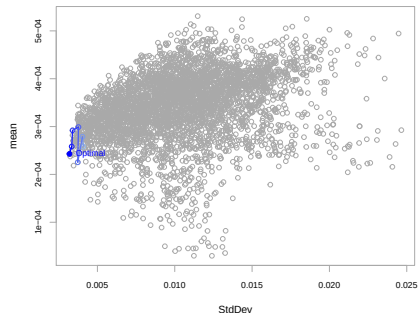
```
> # Add constraints
> portf_maxSR <- add.constraint(
+   portfolio=portfi, ## Initial portfolio
+   type="weightsum", ## Constraint sum weights
+   min_sum=0.9, max_sum=1.1)
> # Add constraints
> portf_maxSR <- add.constraint(
+   portfolio=portf_maxSR,
+   type="long_only") ## box constraint min=0, max=1
> # Add objectives
> portf_maxSR <- add.objective(
+   portfolio=portf_maxSR,
+   type="return", ## Maximize mean return
+   name="mean")
> # Add objectives
> portf_maxSR <- add.objective(
+   portfolio=portf_maxSR,
+   type="risk", ## Minimize stdev
+   name="stdev")
```

Portfolio Optimization

```

> # Perform optimization of weights
> maxSR_DEOpt <- optimize.portfolio(
+   R=rutils::etfenv$returns[, symbolv], ## Specify returns
+   portfolio=portf_maxSR, ## Specify portfolio
+   optimize_method="DEoptim", ## Use DEoptim
+   maxSR=TRUE, ## Maximize Sharpe
+   trace=TRUE, traceDE=0)
> # Plot optimization
> chart.RiskReward(maxSR_DEOpt,
+   risk.col="stdev",
+   return.col="mean")
> maxSR_DEOpt$weights
> maxSR_DEOpt$objective_measures$mean[1]
> maxSR_DEOpt$objective_measures$stdev[[1]]

```

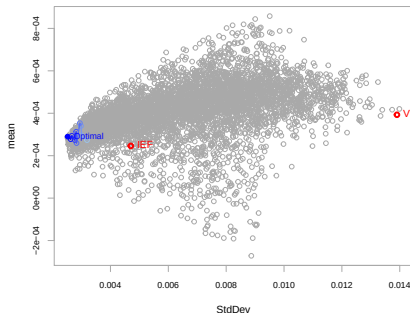


Portfolio Optimization Scatterplot

```

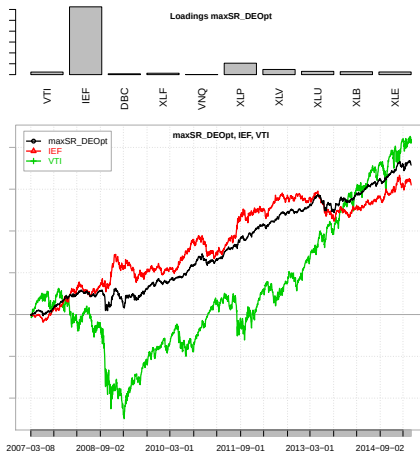
> # Plot optimization
> chart.RiskReward(maxSR_DEOpt,
+   risk.col="stdev",
+   return.col="mean")
>
> # Plot risk/ret points in portfolio scatterplot
> risk_ret_points <- function(rets=rutils::etfenv$returns,
+   risk=c("sd", "ETL"), symbol=c("VTI", "IEF")) {
+   risk <- match.arg(risk) ## Match to arg list
+   if (risk=="ETL") {
+     stopifnot(
+       "package:PerformanceAnalytics" %in% search() ||
+       require("PerformanceAnalytics", quietly=TRUE))
+   } ## end if
+   risk <- match.fun(risk) ## Match to function
+   risk_ret <- t(sapply(rets[, symbol],
+     function(xtsv)
+     c(ret=mean(xtsv), risk=abs(risk(xtsv)))))
+   points(x=risk_ret[, "risk"], y=risk_ret[, "ret"],
+     col="red", lwd=3)
+   text(x=risk_ret[, "risk"], y=risk_ret[, "ret"],
+     labels=rownames(risk_ret), col="red",
+     lwd=2, pos=4)
+ } ## end risk_ret_points
>
> risk_ret_points()

```



Optimized Sharpe Portfolio

```
> plot_portf <- function(portfolio,
+   rets_data=rutils::etfenv$returns) {
+   weightv <- portfolio$weights
+   symbolv <- names(weightv)
+   ## Calculate xts of portfolio
+   portf_max <- xts(
+     rets_data[, symbolv] %*% weightv,
+     order.by=zoo::index(rets_data))
+   colnames(portf_max) <-
+     deparse(substitute(portfolio))
+   graph_params <- par(oma=c(1, 0, 1, 0),
+     mgp=c(2, 1, 0), mar=c(2, 1, 2, 1),
+     cex.lab=0.8, cex.axis=1.0,
+     cex.main=0.8, cex.sub=0.5)
+   layout(matrix(c(1,2), 2),
+     widths=c(1,1), heights=c(1,3))
+   barplot(weightv, names.arg=symbolv,
+     las=3, ylab="", xlab="Symbol", main="")
+   title(main=paste("Loadings",
+     colnames(portf_max)), line=(-1))
+   chart.CumReturns(
+     cbind(portf_max, rets_data[, c("IEF", "VTI")]),
+     lwd=2, ylab="", legend.loc="topleft", main="")
+   title(main=paste0(colnames(portf_max),
+     ", IEF, VTI"), line=(-1))
+   par(graph_params) ## restore original parameters
+   invisible(portf_max)
+ } ## end plot_portf
> maxSR_DEOpt_xts <- plot_portf(portfolio=maxSR_DEOpt)
```



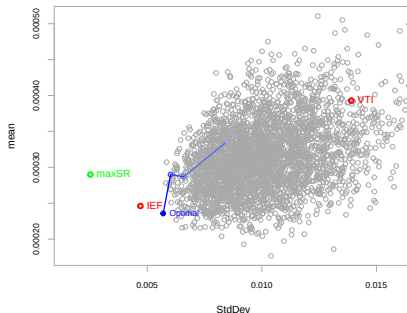
Portfolio Leverage Constraints

The leverage constraint applies to the sum of absolute weights.

```
> # Add leverage constraint abs(weightvum)
> portf_maxSRN <- add.constraint(
+   portfolio=portfi, type="leverage",
+   min_sum=0.9, max_sum=1.1)
> # Add box constraint long/short
> portf_maxSRN <- add.constraint(
+   portfolio=portf_maxSRN,
+   type="box", min=-0.2, max=0.2)
>
> # Add objectives
> portf_maxSRN <- add.objective(
+   portfolio=portf_maxSRN,
+   type="return", ## Maximize mean return
+   name="mean")
> # Add objectives
> portf_maxSRN <- add.objective(
+   portfolio=portf_maxSRN,
+   type="risk", ## Minimize stdev
+   name="stdev")
```

Portfolio Leverage Constraint Optimization

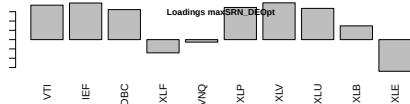
```
> # Perform optimization of weights
> maxSRN_DEOpt <- optimize.portfolio(
+   R=rutils::etfenv$returns[, symbolv], ## Specify returns
+   portfolio=portf_maxSRN, ## Specify portfolio
+   optimize_method="DEoptim", ## Use DEoptim
+   maxSR=TRUE, ## Maximize Sharpe
+   trace=TRUE, traceDE=0)
> # Plot optimization
> chart.RiskReward(maxSRN_DEOpt,
+   risk.col="stdev",
+   return.col="mean",
+   xlim=c(
+     maxSRN_DEOpt$objective_measures$stdev[[1]]-0.001,
+     0.016))
> points(x=maxSRN_DEOpt$objective_measures$stdev[[1]],
+   y=maxSRN_DEOpt$objective_measures$mean[1],
+   col="green", lwd=3)
> text(x=maxSRN_DEOpt$objective_measures$stdev[[1]],
+   y=maxSRN_DEOpt$objective_measures$mean[1],
+   labels="maxSR", col="green",
+   lwd=2, pos=4)
> # Plot risk/ret points in portfolio scatterplot
> risk_ret_points()
```



```
> maxSRN_DEOpt$weights
> maxSRN_DEOpt$objective_measures$mean[1]
> maxSRN_DEOpt$objective_measures$stdev[[1]]
```


Optimized Leverage Constraint Portfolio

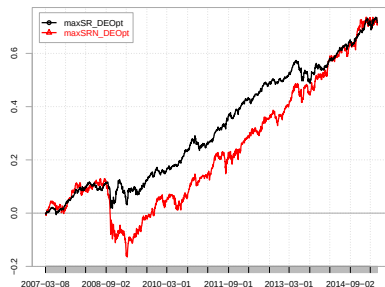
```
> maxSRN_DEOpt_xts <- plot_portf(portfolio=maxSRN_DEOpt)
```



Sharpe Portfolios CumReturns Plots

`chart.CumReturns()` plots the cumulative returns of a time series of returns.

```
> chart.CumReturns(
+   cbind(maxSR_DEOpt_xts, maxSRN_DEOpt_xts),
+   lwd=2, ylab="",
+   legend.loc="topleft", main="")
> rbind(maxSR_DEOpt$weights, maxSRN_DEOpt$weightv)
> c(maxSR_DEOpt$objective_measures$mean,
+   maxSRN_DEOpt$objective_measures$mean)
> c(maxSR_DEOpt$objective_measures$stddev[[1]],
+   maxSRN_DEOpt$objective_measures$stddev[[1]])
```



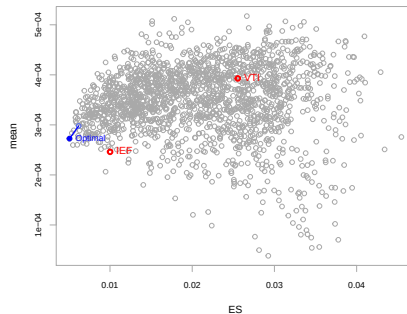
STARR Portfolio Constraints

The objective constraint applies to risk or return.

```
> # Add constraints
> portf_maxSTARR <- add.constraint(
+   portfolio=portfi, ## Initial portfolio
+   type="weightsum", ## Constraint sum weights
+   min_sum=0.9, max_sum=1.1)
> # Add constraints
> portf_maxSTARR <- add.constraint(
+   portfolio=portf_maxSTARR,
+   type="long_only") ## box constraint min=0, max=1
> # Add objectives
> portf_maxSTARR <- add.objective(
+   portfolio=portf_maxSTARR,
+   type="return", ## Maximize mean return
+   name="mean")
> # Add objectives
> portf_maxSTARR <- add.objective(
+   portfolio=portf_maxSTARR,
+   type="risk", ## Minimize Expected Shortfall
+   name="ES")
```

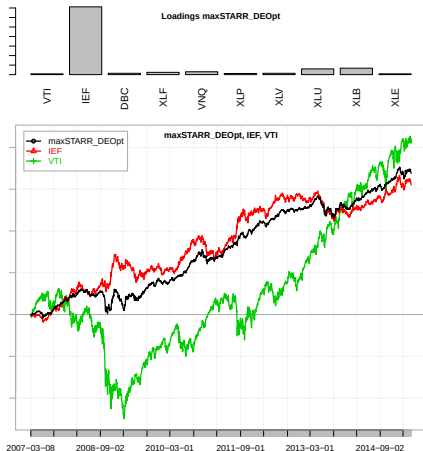
STARR Optimization

```
> # Perform optimization of weights
> maxSTARR_DEOpt <- optimize.portfolio(
+   R=rutils::etfenv$returns[, symbolv], ## Specify returns
+   portfolio=portf_maxSTARR, ## Specify portfolio
+   optimize_method="DEoptim", ## Use DEoptim
+   maxSTARR=TRUE, ## Maximize STARR
+   trace=TRUE, traceDE=0)
>
> # Plot optimization
> chart.RiskReward(maxSTARR_DEOpt,
+   risk.col="ES",
+   return.col="mean")
> # Plot risk/ret points in portfolio scatterplot
> risk_ret_points(risk="ETL")
> maxSTARR_DEOpt$weights
> maxSTARR_DEOpt$objective_measures$mean[1]
> maxSTARR_DEOpt$objective_measures$ES[[1]]
```



Optimized STARR Portfolio

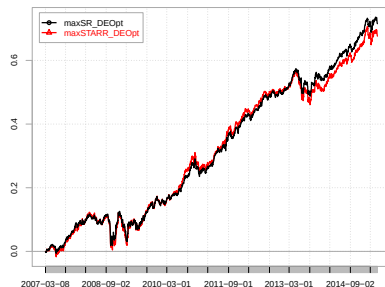
```
> maxSTARR_DEOpt_xts <-  
+   plot_portf(portfolio=maxSTARR_DEOpt)
```



Sharpe STARR CumReturns Plots

`chart.CumReturns()` plots the cumulative returns of a time series of returns.

```
> chart.CumReturns(
+   cbind(maxSR_DEOpt_xts, maxSTARR_DEOpt_xts),
+   lwd=2, ylab="",
+   legend.loc="topleft", main="")
> rbind(maxSR_DEOpt$weights, maxSTARR_DEOpt$weightv)
> c(maxSR_DEOpt$objective_measures$mean,
+   maxSTARR_DEOpt$objective_measures$mean)
> c(maxSR_DEOpt$objective_measures$stddev[[1]],
+   maxSTARR_DEOpt$objective_measures$ES[[1]])
```



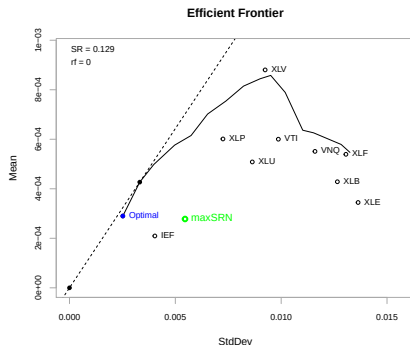
The Efficient Frontier and Capital Market Line

The *efficient frontier* is the set of *efficient portfolios*, that have the lowest risk (standard deviation) for the given level of return.

The Capital Market Line (CML) is the line drawn from the risk-free asset to the tangent point on the Efficient Frontier.

The tangent point on the *efficient frontier* is the *tangency portfolio*.

```
> # Plot the efficient frontier
> chart.EfficientFrontier(maxSR_DEOpt,
+   match.col="stddev",
+   n.portfolios=15, type="l")
> points(x=maxSRN_DEOpt$objective_measures$stddev[[1]],
+   y=maxSRN_DEOpt$objective_measures$mean[1],
+   col="green", lwd=3)
> text(x=maxSRN_DEOpt$objective_measures$stddev[[1]],
+   y=maxSRN_DEOpt$objective_measures$mean[1],
+   labels="maxSRN", col="green",
+   lwd=2, pos=4)
```



minES Portfolio Constraints

The objective constraint applies to risk or return.

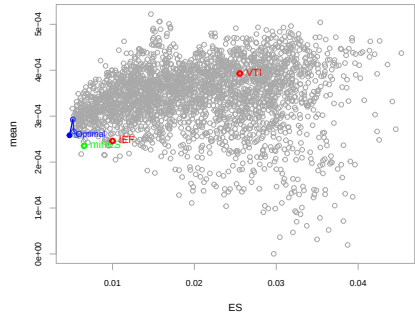
```
> # Add constraints
> portf_minES <- add.constraint(
+   portfolio=portfi, ## Initial portfolio
+   type="weightsum", ## Constraint sum weights
+   min_sum=0.9, max_sum=1.1)
> # Add constraints
> portf_minES <- add.constraint(
+   portfolio=portf_minES,
+   type="long_only") ## box constraint min=0, max=1
> # Add objectives
> portf_minES <- add.objective(
+   portfolio=portf_minES,
+   type="risk", ## Minimize ES
+   name="ES")
```


minES Optimization

```

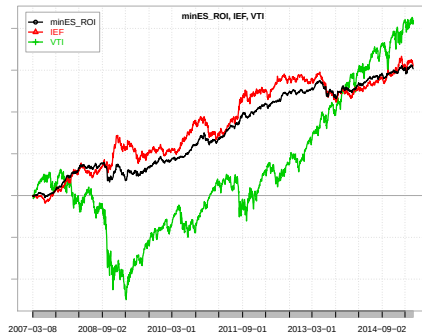
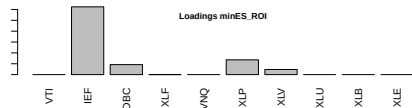
> # Perform optimization of weights
> minESROI <- optimize.portfolio(
+   R=rutils::etfenv$returns[, symbolv], ## Specify returns
+   portfolio=portf_minES, ## Specify portfolio
+   optimize_method="ROI", ## Use ROI
+   trace=TRUE, traceDE=0)
>
> # Plot optimization
> chart.RiskReward(maxSTARR_DEOpt,
+   risk.col="ES",
+   return.col="mean")
> points(x=minESROI$objective_measures$ES[[1]],
+   y=mean(minESROI_xts),
+   col="green", lwd=3)
> text(x=minESROI$objective_measures$ES[[1]],
+   y=mean(minESROI_xts),
+   labels="minES", col="green",
+   lwd=2, pos=4)
> # Plot risk/ret points in portfolio scatterplot
> risk_ret_points(risk="ETL")
> minESROI$weights
> minESROI$objective_measures$ES[[1]]

```



Optimized minES Portfolio

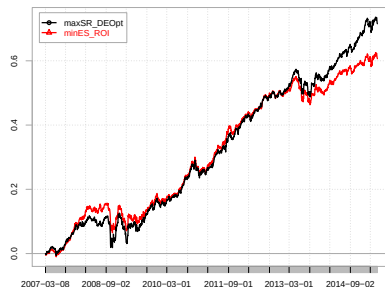
```
> minESROI_xts <-  
+   plot_portf(portfolio=minESROI)
```



Sharpe minES CumReturns Plots

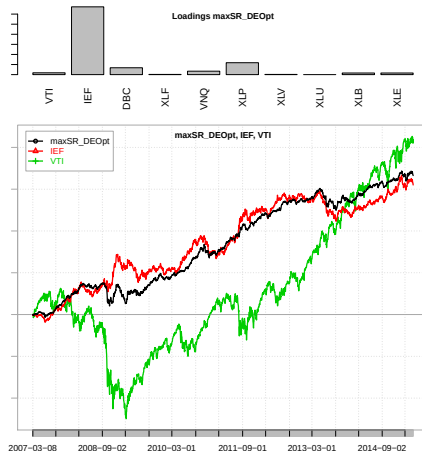
`chart.CumReturns()` plots the cumulative returns of a time series of returns.

```
> chart.CumReturns(
+   cbind(maxSR_DEOpt_xts, minESROI_xts),
+   lwd=2, ylab="",
+   legend.loc="topleft", main="")
> rbind(maxSR_DEOpt$weights, minESROI$weightv)
> c(maxSR_DEOpt$objective_measures$mean,
+   minESROI$objective_measures$mean)
> c(maxSR_DEOpt$objective_measures$stddev[[1]],
+   minESROI$objective_measures$ES[[1]])
```



Out-of-sample Portfolios

```
> # Perform optimization of weights
> maxSR_DEOpt <- optimize.portfolio(
+   R=rutils::etfenv$returns["/2011", symbolv],
+   portfolio=portf_maxSR, ## Specify portfolio
+   optimize_method="DEoptim", ## Use DEoptim
+   maxSR=TRUE, ## Maximize Sharpe
+   trace=TRUE, traceDE=0)
> weights1h <- maxSR_DEOpt$weights
>
> # Plot optimization
> maxSR_DEOpt_xts <-
+   plot_portf(portfolio=maxSR_DEOpt)
```

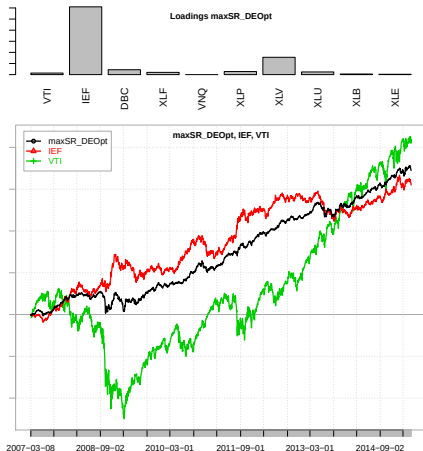


Out-of-sample Portfolios (cont.)

```

> # Perform optimization of weights
> maxSR_DEOpt <- optimize.portfolio(
+   R=rutils::etfenv$returns["2011/", symbolv],
+   portfolio=portf_maxSR, ## Specify portfolio
+   optimize_method="DEoptim", ## Use DEoptim
+   maxSR=TRUE, ## Maximize Sharpe
+   trace=TRUE, traceDE=0)
> weights2h <- maxSR_DEOpt$weights
>
> # Plot optimization
> maxSR_DEOpt_xts <-
+   plot_portf(portfolio=maxSR_DEOpt)

```

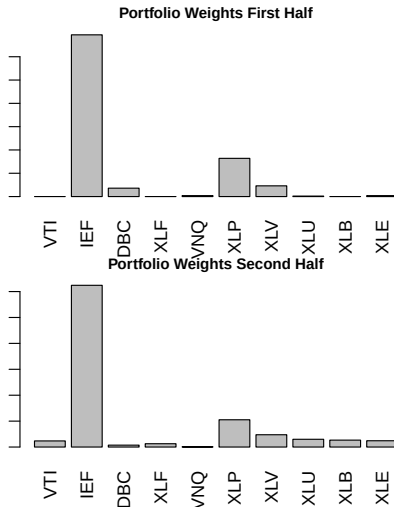


Out-of-sample Portfolio Weights

```

> weights1h
> weights2h
> weights1h - weights2h
> barplot(weights1h,
+   names.arg=names(weights1h),
+   las=3, ylab="", xlab="",
+   main="Portfolio Weights First Half")
> barplot(weights2h,
+   names.arg=names(weights2h),
+   las=3, ylab="", xlab="",
+   main="Portfolio Weights Second Half")

```



Simulating Single-period Defaults

Consider a portfolio of credit assets (bonds or loans) over a single period of time.

At the end of the period, some of the assets default, while the rest don't.

The default probabilities are equal to p_i .

Individual defaults can be simulated by comparing the probabilities p_i with the uniform random numbers u_i .

Default occurs if u_i is less than the default probability p_i :

$$u_i < p_i$$

Simulations in R can be accelerated by pre-computing a vector of random numbers, instead of generating them one at a time in a loop.

Vectors of random numbers allow using *vectorized* functions, instead of inefficient (slow) `for()` loops.

```
> # Calculate random default probabilities
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> nbonds <- 100
> probv <- runif(nbonds, max=0.2)
> mean(probv)
> # Simulate number of defaults
> unifv <- runif(nbonds)
> sum(unifv < probv)
> # Simulate average number of defaults using for() loop (inefficient)
> nsimu <- 1000
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> defaultv <- numeric(nsimu)
> for (i in 1:nsimu) { ## Perform loop
+   unifv <- runif(nbonds)
+   defaultv[i] <- sum(unifv < probv)
+ } ## end for
> # Calculate average number of defaults
> mean(defaultv)
> # Simulate using vectorized functions (efficient way)
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> unifm <- matrix(runif(nsimu*nbonds), ncol=nsimu)
> defaultv <- colSums(unifm < probv)
> mean(defaultv)
> # Plot the distribution of defaults
> x11(width=6, height=5)
> plot(density(defaultv), main="Distribution of Defaults",
+      xlab="number of defaults", ylab="frequency")
> abline(v=mean(defaultv), lwd=3, col="red")
```

Asset Values and Default Thresholds

Defaults can also be simulated using normally distributed variables a_i called *asset values*, instead of the uniformly distributed variables u_i .

The asset values a_i are the *quantiles* corresponding to the uniform variables u_i : $a_i = \Phi^{-1}(u_i)$ (where $\Phi()$ is the cumulative *Standard Normal* distribution).

Similarly, the default probabilities p_i are also transformed into *default thresholds* t_i , which are the *quantiles*: $t_i = \Phi^{-1}(p_i)$.

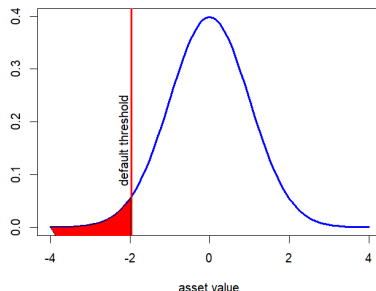
Before, default occurred if u_i was less than the default probability p_i : $u_i < p_i$.

Now, default occurs if the *asset value* a_i is less than the *default threshold* t_i : $a_i < t_i$.

The asset values a_i are mathematical variables which can be negative, so they are not actual company asset values.

```
> # Calculate default thresholds and asset values
> threshv <- qnorm(probv)
> assetm <- qnorm(unifm)
> # Simulate defaults
> defaultv <- colSums(assetm < threshv)
> mean(defaultv)
```

Distribution of Asset Values



```
> # Plot Standard Normal distribution
> x11(width=6, height=5)
> xlim <- 4; threshv <- qnorm(0.025)
> curve(expr=dnorm(x), type="l", xlim=c(-xlim, xlim),
+ xlab="asset value", ylab="", lwd=3,
+ col="blue", main="Distribution of Asset Values")
> abline(v=threshv, col="red", lwd=3)
> text(x=threshv-0.1, y=0.15, labels="default threshold",
+ lwd=2, srt=90, pos=3)
> # Plot polygon area
> xvar <- seq(-xlim, xlim, length=100)
> yvar <- dnorm(xvar)
> intail <- ((xvar >= (-xlim)) & (xvar <= threshv))
> polygon(c(xlim, xvar[intail], threshv),
+ c(-1, yvar[intail], -1), col="red")
```


Vasicek Model of Correlated Asset Values

So far, the asset values are independent from each other, but in reality default events are correlated.

The *Vasicek* model introduces correlation between the asset values a_i .

Under the *Vasicek* single factor model, the asset value a_i is equal to the sum of a *systematic* factor s , plus an *idiosyncratic* factor z_i :

$$a_i = \sqrt{\rho} s + \sqrt{1 - \rho} z_i$$

Where ρ is the correlation between asset values.

The variables s , z_i , and a_i all follow the *Standard Normal* distribution $\phi(0, 1)$.

The matrix of asset values is arranged with columns corresponding to simulations and rows corresponding to bonds or loans.

The *Vasicek* model resembles the *CAPM* model, with the asset value equal to the sum of a *systematic* factor plus an *idiosyncratic* factor.

The Bank for International Settlements (BIS) uses the *Vasicek* model as part of its regulatory capital requirements for bank credit risk:

http://bis2information.org/content/Vasicek_model

<https://www.bis.org/bcbs/basel3.htm>

<https://www.bis.org/bcbs/irbriskweight.pdf>

```
> # Define correlation parameters
> rho <- 0.2
> rhos <- sqrt(rho) ; rhosm <- sqrt(1-rho)
> nbonds <- 5 ; nsimu <- 10000
> # Calculate vector of systematic and idiosyncratic factors
> sysv <- rnorm(nsimu)
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> isync <- rnorm(nsimu*nbonds)
> dim(isync) <- c(nbonds, nsimu)
> # Simulate asset values using vectorized functions (efficient way)
> assetm <- t(rhos*sysv + t(rhosm*isync))
> # Asset values are standard normally distributed
> apply(assetm, MARGIN=1, function(x) c(mean=mean(x), sd=sd(x)))
> # Calculate correlations between asset values
> cor(t(assetm))
> # Simulate asset values using for() loop (inefficient way)
> # Allocate matrix of assets
> assetn <- matrix(nrow=nbonds, ncol=nsimu)
> # Simulate asset values using for() loop
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> for (i in 1:nsimu) { ## Perform loop
+   assetn[, i] <- rhos*sysv[i] + rhosm*rnorm(nbonds)
+ } ## end for
> all.equal(assetn, assetm)
> # benchmark the speed of the two methods
> library(microbenchmark)
> summary(microbenchmark(
+   forloop={for (i in 1:nsimu) {
+     rhos*sysv[i] + rhosm*rnorm(nbonds)},
+   vectorized={t(rhos*sysv + t(rhosm*isync))},
+   times=10) [, c(1, 4, 5)])
```

Vasicek Model of Correlated Defaults

Under the *Vasicek* model, default occurs if the *asset value* a_i is less than the *default threshold* t_i :

$$a_i = \sqrt{\rho}s + \sqrt{1 - \rho}z_i$$

$$a_i < t_i$$

The *systematic* factor s may be considered to represent the state of the macro economy, with positive values representing an economic expansion, and negative values representing an economic recession.

When the value of the *systematic* factor s is positive, then the asset values will all tend to be bigger as well, which will produce fewer defaults.

But when the *systematic* factor is negative, then the asset values will tend to be smaller, which will produce more defaults.

This way the *Vasicek* model introduces a correlation among defaults.

```
> # Calculate random default probabilities
> nbonds <- 5
> probv <- runif(nbonds, max=0.2)
> mean(probv)
> # Calculate default thresholds
> threshv <- qnorm(probv)
> # Calculate number of defaults using vectorized functions (efficient)
> # Calculate vector of number of defaults
> rowMeans(assetm < threshv)
> probv
> # Calculate number of defaults using for() loop (inefficient way)
> # Allocate matrix of defaultm
> defaultm <- matrix(nrow=nbonds, ncol=nsimu)
> # Simulate asset values using for() loop
> for (i in 1:nsimu) { ## Perform loop
+   defaultm[, i] <- (assetm[, i] < threshv)
+ } ## end for
> rowMeans(defaultm)
> rowMeans(assetm < threshv)
> # Calculate correlations between defaults
> cor(t(defaultm))
```

Asset Correlation and Default Correlation

Default correlation is defined as the correlation between the Boolean vectors of default events.

The *Vasicek* model introduces correlation among default events, through the correlation of *asset values*.

If *asset values* have a positive correlation, then the defaults among credits are clustered together, and if one credit defaults then the other credits are more likely to default as well.

Empirical studies have found that the asset correlation ρ can vary between 5% to 20%, depending on the default risk.

Credits with higher default risk tend to also have higher asset correlation, since they are more sensitive to the economic conditions.

Default correlations are usually much lower than the corresponding asset correlations.

```
> # Define default probabilities
> nbonds <- 2
> defprob <- 0.2
> threshv <- qnorm(defprob)
> # Define correlation parameters
> rho <- 0.2
> rhos <- sqrt(rho) ; rhosm <- sqrt(1-rho)
> # Calculate vector of systematic factors
> nsimu <- 1000
> sysv <- rnorm(nsimu)
> isync <- rnorm(nsimu*nbonds)
> dim(isync) <- c(nbonds, nsimu)
> # Simulate asset values using vectorized functions
> assetm <- t(rhos*sysv + t(rhosm*isync))
> # Calculate number of defaults using vectorized functions
> defaultm <- (assetm < threshv)
> # Calculate average number of defaults and compare to defprob
> rowMeans(defaultm)
> defprob
> # Calculate correlations between assets
> cor(t(assetm))
> # Calculate correlations between defaults
> cor(t(defaultm))
```

Cumulative Defaults Under the Vasicek Model

A formula for the default distribution under the Vasicek Model can be derived under the assumption that the number of assets is very large and that they all have the same default probabilities $p_i = p$.

In that case the single default threshold is equal to $t = \Phi^{-1}(p)$.

If the systematic factor s is fixed, then the *asset value* a_i follows the *Normal* distribution with mean equal to $\sqrt{\rho}s$ and standard deviation equal to $\sqrt{1-\rho}$:

$$a_i = \sqrt{\rho}s + \sqrt{1-\rho}z_i$$

The conditional default probability $p(s)$, given the systematic factor s , is equal to:

$$p(s) = \Phi\left(\frac{t - \sqrt{\rho}s}{\sqrt{1-\rho}}\right)$$

Since the systematic factor s is fixed, then the defaults are all independent with the same default probability $p(s)$.

Because the number of assets is very large, the percentage x of the portfolio that defaults, is equal to the conditional default probability $x = p(s)$.

We can invert the formula $x = \Phi\left(\frac{t - \sqrt{\rho}s}{\sqrt{1-\rho}}\right)$ to obtain the systematic factor s :

$$s = \frac{\sqrt{1-\rho}\Phi^{-1}(x) - t}{\sqrt{\rho}}$$

Since the systematic factor s follows the *Standard Normal* distribution, then the portfolio cumulative default probability $P(x)$ is equal to:

$$P(x) = \Phi(s) = \Phi\left(\frac{\sqrt{1-\rho}\Phi^{-1}(x) - t}{\sqrt{\rho}}\right)$$

Cumulative Default Distribution And Correlation

The cumulative portfolio default probability $P(x)$:

$$P(x) = \Phi\left(\frac{\sqrt{1-\rho}\Phi^{-1}(x) - t}{\sqrt{\rho}}\right)$$

Depends on the correlation parameter ρ .

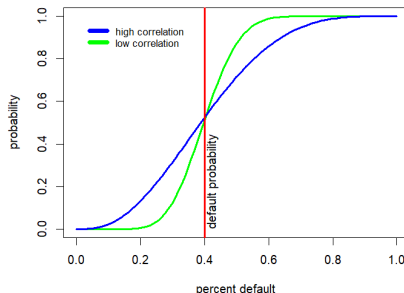
If the correlation ρ is very low (close to 0) then the percentage x of the portfolio defaults is always very close to the default probability p , and the cumulative default probability curve is steep close to the expected value of p .

If the correlation ρ is very high (close to 1) then the percentage x of the portfolio defaults has a very wide dispersion around the default probability p , and the cumulative default probability curve is flat close to the expected value of p .

This is because with high correlation, the assets will tend to all default together or not default.

```
> # Define cumulative default distribution function
> cumdefdistr <- function(x, threshv=(-2), rho=0.2)
+   pnorm((sqrt(1-rho)*qnorm(x) - threshv)/sqrt(rho))
> defprob <- 0.4; threshv <- qnorm(defprob)
> cumdefdistr(x=0.2, threshv=qnorm(defprob), rho=rho)
> # Plot cumulative default distribution function
> curve(expr=cumdefdistr(x, threshv=threshv, rho=0.05),
+ xlim=c(0, 0.999), lwd=3, xlab="percent default", ylab="probability",
+ col="green", main="Cumulative Default Probabilities")
```

Cumulative Default Probabilities



```
> # Plot default distribution with higher correlation
> curve(expr=cumdefdistr(x, threshv=threshv, rho=0.2),
+ xlim=c(0, 0.999), add=TRUE, lwd=3, col="blue", main="")
> # Add legend
> legend(x="topleft",
+ legend=c("high correlation", "low correlation"),
+ title=NULL, inset=0.05, cex=1.0, bg="white",
+ bty="n", lwd=6, lty=1, col=c("blue", "green"))
> # Add unconditional default probability
> abline(v=defprob, col="red", lwd=3)
> text(x=defprob, y=0.0, labels="default probability",
+ lwd=2, srt=90, pos=4)
```

Distribution of Defaults Under the Vasicek Model

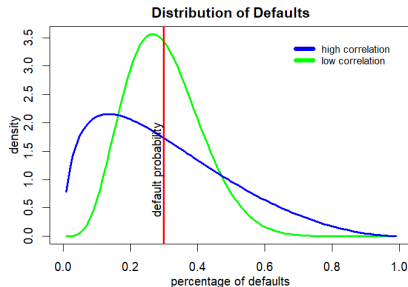
The probability density $f(x)$ of portfolio defaults is equal to the derivative of the cumulative default distribution $P(x)$:

$$f(x) = \frac{\sqrt{1-\rho}}{\sqrt{\rho}} \exp\left(-\frac{1}{2\rho}(\sqrt{1-\rho}\Phi^{-1}(x) - t)^2\right) + \frac{1}{2}\Phi^{-1}(x)^2$$

If the correlation ρ is very low (close to 0) then the probability density $f(x)$ is centered around the default probability p .

If the correlation ρ is very high (close to 1) then the probability density $f(x)$ is wide, with significant probability of large portfolio defaults and also small portfolio defaults.

```
> # Define default probability density function
> defdistr <- function(x, threshv=(-2), rho=0.2)
+   sqrt((1-rho)/rho)*exp(-(sqrt(1-rho)*qnorm(x) -
+   threshv)^2/(2*rho) + qnorm(x)^2/2)
> # Define parameters
> rho <- 0.2 ; rhos <- sqrt(rho) ; rhosm <- sqrt(1-rho)
> defprob <- 0.3; threshv <- qnorm(defprob)
> defdistr(0.03, threshv=threshv, rho=rho)
> # Plot probability distribution of defaults
> curve(expr=defdistr(x, threshv=threshv, rho=0.1),
+ xlab="Default percentage", ylab="Density",
+ col="green", main="Distribution of Defaults")
```



```
> # Plot default distribution with higher correlation
> curve(expr=defdistr(x, threshv=threshv, rho=0.3),
+ xlab="default percentage", ylab="",
+ add=TRUE, lwd=3, col="blue", main="")
> # Add legend
> legend(x="topright",
+ legend=c("high correlation", "low correlation"),
+ title=NULL, inset=0.05, cex=1.0, bg="white",
+ bty="n", lwd=6, lty=1, col=c("blue", "green"))
> # Add unconditional default probability
> abline(v=defprob, col="red", lwd=3)
> text(x=defprob, y=2, labels="default probability",
+ lwd=2, srt=90, pos=2)
```

Distribution of Defaults Under Extreme Correlations

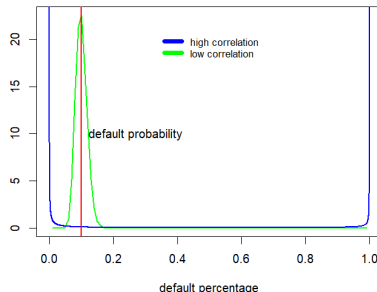
If the correlation ρ is close to 0, then the asset values a_i are independent from each other, and defaults are also independent, so that the percentage of portfolio defaults is very close to the default probability p .

In that case, the probability density of portfolio defaults is very narrow and is centered on the default probability p .

If the correlation ρ is close to 1, then the asset values a_i are almost the same, and defaults occur at the same time, so that the percentage of portfolio defaults is either 0 or 1.

In that case, the probability density of portfolio defaults becomes *bimodal*, with two peaks around zero and 1.

Distribution of Defaults



```
> # Plot default distribution with low correlation
> curve(expr=defdistr(x, threshv=threshv, rho=0.01),
+       xlab="default percentage", ylab="", lwd=2,
+       col="green", main="Distribution of Defaults")
> # Plot default distribution with high correlation
> curve(expr=defdistr(x, threshv=threshv, rho=0.99),
+       xlab="percentage of defaults", ylab="density",
+       add=TRUE, lwd=2, n=10001, col="blue", main="")
```

```
> # Add legend
> legend(x="top", legend=c("high correlation", "low correlation"),
+       title=NULL, inset=0.1, cex=1.0, bg="white",
+       bty="n", lwd=6, lty=1, col=c("blue", "green"))
> # Add unconditional default probability
> abline(v=0.1, col="red", lwd=2)
> text(x=0.1, y=10, lwd=2, pos=4, labels="default probability")
```

Numerical Integration of Functions

The function `integrate()` performs numerical integration of a function of a single variable, i.e. it calculates a definite integral over an integration interval.

Additional parameters can be passed to the integrated function through the dots `"..."` argument of the function `integrate()`.

The function `integrate()` accepts the integration limits `-Inf` and `Inf` equal to minus and plus infinity.

```
> # Get help for integrate()
> ?integrate
> # Calculate slowly converging integral
> func <- function(x) {1/((x+1)*sqrt(x))}
> integrate(func, lower=0, upper=10)
> integrate(func, lower=0, upper=Inf)
> # Integrate function with parameter lambdaf
> func <- function(x, lambdaf=1) {
+   exp(-x*lambdaf)
+ } ## end func
> integrate(func, lower=0, upper=Inf)
> integrate(func, lower=0, upper=Inf, lambdaf=2)
> # Cumulative probability over normal distribution
> pnorm(-2)
> integrate(dnorm, low=2, up=Inf)
> str(dnorm)
> pnorm(-1)
> integrate(dnorm, low=2, up=Inf, mean=1)
> # Expected value over normal distribution
> integrate(function(x) x*dnorm(x), low=2, up=Inf)
```


Portfolio Loss Distribution

The expected loss (EL) of a credit portfolio is equal to the sum of the default probabilities p_i multiplied by the loss given default LGD (aka the *loss severity* - equal to 1 minus the *recovery rate*):

$$EL = \sum_{i=1}^n p_i LGD_i$$

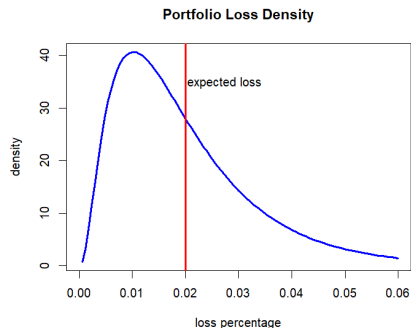
Then the *cumulative loss distribution* is equal to:

$$P(x) = \Phi\left(\frac{\sqrt{1-\rho}\Phi^{-1}\left(\frac{x}{LGD}\right) - t}{\sqrt{\rho}}\right)$$

And the *default distribution* is the derivative, and is equal to:

$$f(x) = \frac{\sqrt{1-\rho}}{LGD\sqrt{\rho}} \exp\left(-\frac{1}{2\rho}\left(\sqrt{1-\rho}\Phi^{-1}\left(\frac{x}{LGD}\right) - t\right)^2 + \frac{1}{2}\Phi^{-1}\left(\frac{x}{LGD}\right)^2\right)$$

```
> # Vasicek model parameters
> rho <- 0.1; lgd <- 0.4
> defprob <- 0.05; threshv <- qnorm(defprob)
> # Define Vasicek cumulative loss distribution
> cumlossdistr <- function(x, threshv=(-2), rho=0.2, lgd=0.4)
+   pnorm((sqrt(1-rho)*qnorm(x/lgd) - threshv)/sqrt(rho))
> # Define Vasicek loss distribution function
> lossdistr <- function(x, threshv=(-2), rho=0.2, lgd=0.4)
+   sqrt((1-rho)/rho)*exp(-(sqrt(1-rho)*qnorm(x/lgd) - threshv)^2/(2*rho) + qnorm(x/lgd)^2/2)/lgd
```



```
> # Plot probability distribution of losses
> x11(width=6, height=5)
> curve(expr=lossdistr(x, threshv=threshv, rho=rho),
+ cex.main=1.8, cex.lab=1.8, cex.axis=1.5,
+ type="l", xlim=c(0, 0.06),
+ xlab="loss percentage", ylab="density", lwd=3,
+ col="blue", main="Portfolio Loss Density")
> # Add line for expected loss
> abline(v=lgd*defprob, col="red", lwd=3)
> text(x=lgd*defprob-0.001, y=35, labels="expected loss", lwd=3, pos=4)
```

Collateralized Debt Obligations (CDOs)

Collateralized Debt Obligations (cash CDOs) are securities (bonds) collateralized by other debt assets.

The CDO assets can be debt instruments like bonds, loans, and mortgages.

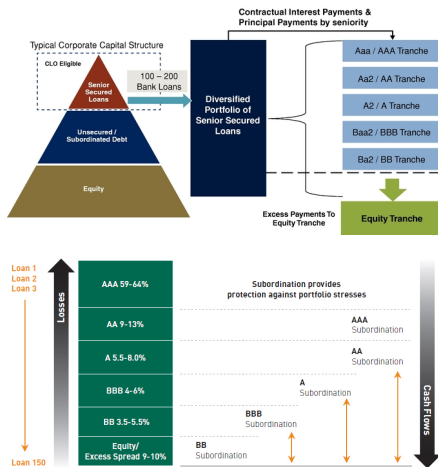
The CDO liabilities are CDO tranches, which receive cashflows from the CDO assets, and are exposed to their defaults.

CDO tranches have an attachment point (subordination, i.e. the percentage of asset default losses at which the tranche starts absorbing those losses), and a detachment point when the tranche is wiped out (suffers 100% losses).

The *equity tranche* is the most junior tranche, and is the first to absorb default losses.

The *mezzanine tranches* are senior to the *equity tranche* and absorb losses only after the *equity tranche* is wiped out.

The *senior tranche* is the most senior tranche, and is the last to absorb losses.



CDO Tranche Losses

Single-tranche (synthetic) CDOs are credit default swaps which reference credit portfolios.

The expected loss EL on a CDO tranche is:

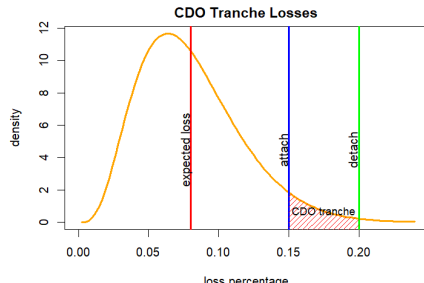
$$EL = \int_a^d \frac{x-a}{d-a} f(x) dx + \int_d^{LGD} f(x) dx$$

Where $f(x)$ is the density of portfolio losses, and a and d are the tranche attachment (subordination) and detachment points.

The difference $(d - a)$ is the tranche *thickness*, so that EL is the expected loss as a percentage of the tranche notional.

A single-tranche CDO can be thought of as a short option spread on the asset defaults, struck at the attachment and detachment points.

```
> # Define Vasicek cumulative loss distribution
> # (with error handling for x)
> cumlossdistr <- function(x, threshv=(-2), rho=0.2, lgd=0.4) {
+   qnormv <- ifelse(x/lgd < 0.999, qnorm(x/lgd), 3.1)
+   pnorm((sqrt(1-rho)*qnormv - threshv)/sqrt(rho))
+ } ## end cumlossdistr
> # Define Vasicek loss distribution function
> # (vectorized version with error handling for x)
> lossdistr <- function(x, threshv=(-2), rho=0.1, lgd=0.4) {
+   qnormv <- ifelse(x/lgd < 0.999, qnorm(x/lgd), 3.1)
+   sqrt((1-rho)/rho)*exp(-(sqrt(1-rho)*qnormv - threshv)^2/(2*rho))
+ } ## end lossdistr
```



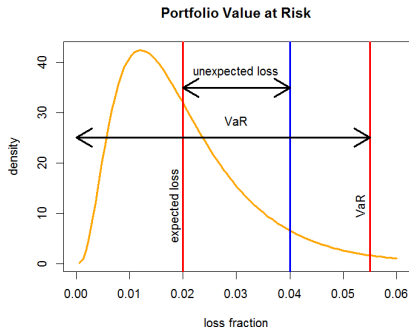
```
> defprob <- 0.2; threshv <- qnorm(defprob)
> rho <- 0.1; lgd <- 0.4
> attachp <- 0.15; detachp <- 0.2
> # Expected tranche loss is sum of two terms
> tranchel <-
+   ## Loss between attachp and detachp
+   integrate(function(x) (x-attachp)*
+     lossdistr(x, threshv=threshv, rho=rho, lgd=lgd),
+     low=attachp, up=detachp)$value/(detachp-attachp) +
+   ## Loss in excess of detachp
+   (1-cumlossdistr(x=detachp, threshv=threshv, rho=rho, lgd=lgd))
> # Plot probability distribution of losses
> curve(expr=lossdistr(x, threshv=threshv, rho=rho),
+   cex.main=1.8, cex.lab=1.8, cex.axis=1.5,
+   type="l", xlim=c(0, 3*lgd*defprob),
+   xlab="loss percentage", ylab="density", lwd=3,
+   col="orange", main="CDO Tranche Losses")
> # Add line for expected loss
> abline(v=lgd*defprob, col="red", lwd=3)
> text(x=lgd*defprob-0.001, y=4, labels="expected loss",
```

Portfolio Value at Risk

Value at Risk (VaR) measures extreme portfolio loss (but not the worst possible loss), defined as the *quantile* of the loss distribution, corresponding to a given confidence level α .

A loss exceeding the EL is called the Unexpected Loss (UL), and can be calculated from the *portfolio loss distribution*.

```
> # Add lines for unexpected loss
> abline(v=0.04, col="blue", lwd=3)
> arrows(x0=0.02, y0=35, x1=0.04, y1=35, code=3, lwd=3, cex=0.5)
> text(x=0.03, y=36, labels="unexpected loss", lwd=2, pos=3)
> # Add lines for VaR
> abline(v=0.055, col="red", lwd=3)
> arrows(x0=0.0, y0=25, x1=0.055, y1=25, code=3, lwd=3, cex=0.5)
> text(x=0.03, y=26, labels="VaR", lwd=2, pos=3)
> text(x=0.055-0.001, y=10, labels="VaR", lwd=2, srt=90, pos=3)
```



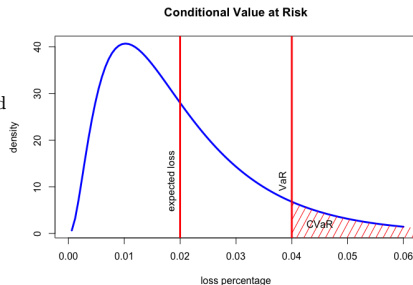
Portfolio Conditional Value at Risk

The *Conditional Value at Risk (CVaR)* is equal to the average of the *VaR* values for all the confidence levels greater than the given confidence level α :

$$CVaR = \frac{1}{1 - \alpha} \int_{\alpha}^1 VaR(p) dp = \frac{1}{1 - \alpha} \int_{VaR}^{LGD} x f(x) dx$$

The *Conditional Value at Risk* is also called the Expected Shortfall (*ES*), or Expected Tail Loss (*ETL*).

```
> varisk <- 0.04; varmax <- 4*lgd*defprob
> # Calculate CVaR
> cvar <- integrate(function(x) x*lossdistr(x, threshv=threshv,
+   rho=rho, lgd=lgd), low=varisk, up=lgd)$value
> cvar <- cvar/integrate(lossdistr, low=varisk, up=lgd,
+   threshv=threshv, rho=rho, lgd=lgd)$value
> # Plot probability distribution of losses
> curve(expr=lossdistr(x, threshv=threshv, rho=rho),
+ type="l", xlim=c(0, 0.06),
+ xlab="loss percentage", ylab="density", lwd=3,
+ col="blue", main="Conditional Value at Risk")
> # Add line for expected loss
> abline(v=lgd*defprob, col="red", lwd=3)
> text(x=lgd*defprob-0.001, y=10, labels="expected loss", lwd=2, s:
```



```
> # Add lines for VaR
> abline(v=varisk, col="red", lwd=3)
> text(x=varisk-0.001, y=10, labels="VaR",
+   lwd=2, srt=90, pos=3)
> # Add shading for CVaR
> vars <- seq(varisk, varmax, length=100)
> densv <- sapply(vars, lossdistr,
+   threshv=threshv, rho=rho)
> # Draw shaded polygon
> polygon(c(varisk, vars, varmax), density=20,
+   c(-1, densv, -1), col="red", border=NA)
> text(x=varisk+0.005, y=0, labels="CVaR", lwd=2, pos=3)
```

Value at Risk Under the Vasicek Model

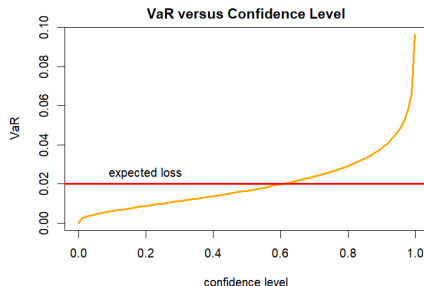
Value at Risk (VaR) measures extreme portfolio loss (but not the worst possible loss), defined as the *quantile* of the loss distribution, corresponding to a given confidence level α .

The *cumulative loss distribution* is equal to:

$$P(x) = \Phi\left(\frac{\sqrt{1-\rho}\Phi^{-1}\left(\frac{x}{LGD}\right) - t}{\sqrt{\rho}}\right)$$

Then the *quantile* of the loss distribution ($x = VaR$), for a given a confidence level $\alpha = P(x)$, is given by the inverse of the *cumulative loss distribution*:

$$VaR(\alpha) = LGD \cdot \Phi\left(\frac{\sqrt{\rho}\Phi^{-1}(\alpha) + t}{\sqrt{1-\rho}}\right)$$

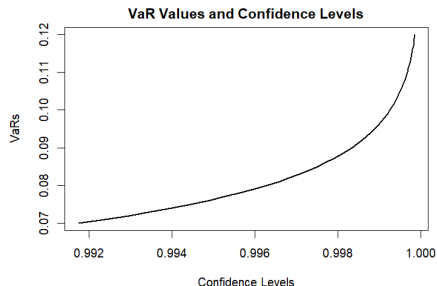


```
> # VaR (quantile of the loss distribution)
> varfun <- function(x, threshv=qnorm(0.1), rho=0.1, lgd=0.4)
+   lgd*pnorm((sqrt(rho)*qnorm(x) + threshv)/sqrt(1-rho))
> varfun(x=0.99, threshv=threshv, rho=rho, lgd=lgd)
> # Plot VaR
> curve(expr=varfun(x, threshv=threshv, rho=rho, lgd=lgd),
+ type="l", xlim=c(0, 0.999), xlab="confidence level", ylab="VaR", lwd=3,
+ col="orange", main="VaR versus Confidence Level")
> # Add line for expected loss
> abline(h=lgd*defprob, col="red", lwd=3)
> text(x=0.2, y=lgd*defprob, labels="expected loss", lwd=2, pos=3)
```

Value at Risk and Confidence Levels

The confidence levels of VaR values can also be calculated by integrating over the tail of the loss density function.

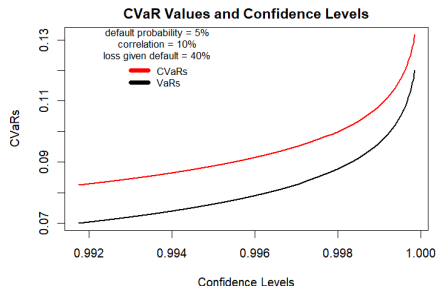
```
> # Integrate lossdistr() over full range
> integrate(lossdistr, low=0.0, up=lgd,
+   threshv=threshv, rho=rho, lgd=lgd)
> # Calculate expected losses using lossdistr()
> integrate(function(x) x*lossdistr(x, threshv=threshv,
+   rho=rho, lgd=lgd), low=0.0, up=lgd)
> # Calculate confidence levels corresponding to VaR values
> vars <- seq(0.07, 0.12, 0.001)
> conv <- sapply(vars, function(varisk) {
+   integrate(lossdistr, low=varisk, up=lgd,
+   threshv=threshv, rho=rho, lgd=lgd)
+ }) ## end sapply
> conv <- cbind(as.numeric(t(conv)[, 1]), vars)
> colnames(conv) <- c("levels", "VaRs")
> # Calculate 95% confidence level VaR value
> conv[match(TRUE, conv[, "levels"] < 0.05), "VaRs"]
> plot(x=1-conv[, "levels"],
+   y=conv[, "VaRs"], lwd=2,
+   xlab="confidence level", ylab="VaRs",
+   t="l", main="VaR Values and Confidence Levels")
```



Conditional Value at Risk Under the Vasicek Model

The $CVaR$ values can be calculated by integrating over the tail of the loss density function.

```
> # Calculate CVaR values
> cvars <- sapply(vars, function(varisk) {
+   integrate(function(x) x*lossdistr(x, threshv=threshv,
+ rho=rho, lgd=lgd), low=varisk, up=lgd)) ## end sapply
> conv <- cbind(conv, as.numeric(t(cvars)[, 1]))
> colnames(conv)[3] <- "CVaRs"
> # Divide CVaR by confidence level
> conv[, "CVaRs"] <- conv[, "CVaRs"]/conv[, "levels"]
> # Calculate 95% confidence level CVaR value
> conv[match(TRUE, conv[, "levels"] < 0.05), "CVaRs"]
> # Plot CVaRs
> plot(x=1-conv[, "levels"], y=conv[, "CVaRs"],
+      t="l", col="red", lwd=2,
+      ylim=range(conv[, c("VaRs", "CVaRs")]),
+      xlab="confidence level", ylab="CVaRs",
+      main="CVaR Values and Confidence Levels")
```



```
> # Add VaRs
> lines(x=1-conv[, "levels"], y=conv[, "VaRs"], lwd=2)
> # Add legend
> legend(x="topleft", legend=c("CVaRs", "VaRs"),
+       title="default probability = 5%",
+       correlation = 10%,
+       loss given default = 40%",
+       inset=0.1, cex=1.0, bg="white", bty="n",
+       lwd=6, lty=1, col=c("red", "black"))
```


Simulating Portfolio Losses Under the Vasicek Model

If the default probabilities p_i are not all the same, then there's no simple formula for the *portfolio loss distribution* under the Vasicek Model.

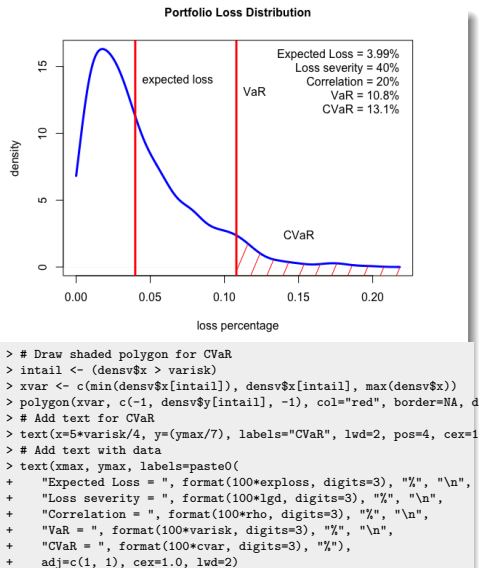
In that case the portfolio losses and VaR must be simulated.

```
> # Define model parameters
> nbonds <- 300; nsimu <- 1000; lgd <- 0.4
> # Define correlation parameters
> rho <- 0.2; rhos <- sqrt(rho); rhosm <- sqrt(1-rho)
> # Calculate default probabilities and thresholds
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> probv <- runif(nbonds, max=0.2)
> threshv <- qnorm(probv)
> # Simulate losses under the Vasicek model
> sysv <- rnorm(nsimu)
> assetm <- matrix(rnorm(nsimu*nbonds), ncol=nsimu)
> assetm <- t(rhos*sysv + t(rhosm*assetm))
> lossv <- lgd*colSums(assetm < threshv)/nbonds
```

VaR and CVaR Under the Vasicek Model

The function `density()` calculates a kernel estimate of the probability density for a sample of data, and returns a list with a vector of loss values and a vector of corresponding densities.

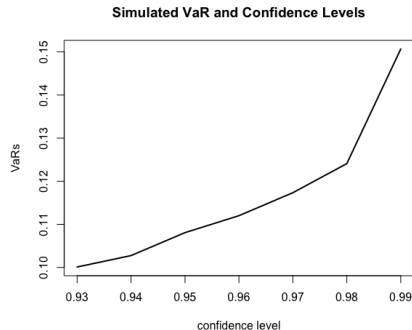
```
> # Calculate VaR from confidence level
> confl <- 0.95
> varisk <- quantile(losssv, confl)
> # Calculate the CVaR as the mean losses in excess of VaR
> cvar <- mean(losssv[losssv > varisk])
> # Plot the density of portfolio losses
> densv <- density(losssv, from=0)
> plot(densv, xlab="loss percentage", ylab="density",
+      cex.main=1.0, cex.lab=1.0, cex.axis=1.0,
+      lwd=3, col="blue", main="Portfolio Loss Distribution")
> # Add vertical line for expected loss
> exploss <- lgd*mean(probv)
> abline(v=exploss, col="red", lwd=3)
> xmax <- max(densv$x); ymax <- max(densv$y)
> text(x=exploss, y=(6*ymax/7), labels="expected loss",
+      lwd=2, pos=4, cex=1.0)
> # Add vertical line for VaR
> abline(v=varisk, col="red", lwd=3)
> text(x=varisk, y=4*ymax/5, labels="VaR", lwd=2, pos=4, cex=1.0)
```



Simulating VaR Under the Vasicek Model

The *VaR* can be calculated from the simulated portfolio losses using the function `quantile()`.

The function `quantile()` calculates the sample quantiles. It uses interpolation to improve the accuracy. Information about the different interpolation methods can be found by typing `?quantile`.



```
> # Calculate VaRs from confidence levels
> conv <- seq(0.93, 0.99, 0.01)
> vars <- quantile(lossv, probs=conv)
> plot(x=conv, y=vars, t="l", lwd=2,
+       xlab="confidence level", ylab="VaRs",
+       main="Simulated VaR and Confidence Levels")
```

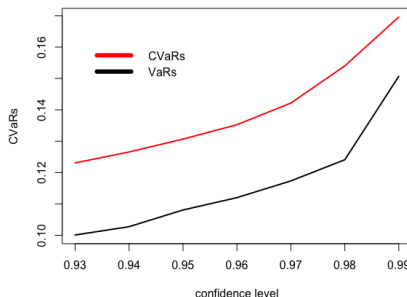
Simulating CVaR Under the Vasicek Model

The CVaR can be calculated from the frequency of tail losses in excess of the VaR.

The function `table()` calculates the frequency distribution of categorical data.

```
> # Calculate CVaRs
> cvars <- sapply(vars, function(varisk) {
+   mean(lossv[lossv >= varisk])
+ }) ## end sapply
> cvars <- cbind(cvars, vars)
> # Alternative CVaR calculation using frequency table
> # first calculate frequency table of losses
> # tablev <- table(lossv)/nsimu
> # Calculate CVaRs from frequency table
> cvars <- sapply(vars, function(varisk) {
+   # tailrisk <- tablev[names(tablev) > varisk]
+   # tailrisk %>% as.numeric(names(tailrisk)) / sum(tailrisk)
+ }) ## end sapply
```

Simulated CVaR and Confidence Levels



```
> # Plot CVaRs
> plot(x=conv, y=cvars[, "cvars"],
+   t="l", col="red", lwd=2, ylim=range(cvars),
+   xlab="confidence level", ylab="CVaRs",
+   main="Simulated CVaR and Confidence Levels")
> # Add VaRs
> lines(x=conv, y=cvars[, "vars"], lwd=2)
> # Add legend
> legend(x="topleft", legend=c("CVaRs", "VaRs"), bty="n",
+   title=NULL, inset=0.05, cex=1.0, bg="white",
+   y.intersp=0.3, lwd=6, lty=1, col=c("red", "black"))
```

Function for Simulating VaR Under the Vasicek Model

The function `calc_var()` simulates default losses under the *Vasicek* model, for a vector of confidence levels, and calculates a vector of *VaR* and *CVaR* values.

```
> calc_var <- function(threshv, ## Default thresholds
+   lgd=0.6, ## loss given default
+   rhos, rhosm, ## asset correlation
+   nsimu=1000, ## number of simulations
+   conv=seq(0.93, 0.99, 0.01) ## Confidence levels
+ ) {
+   ## Define model parameters
+   nbonds <- NROW(threshv)
+   ## Simulate losses under the Vasicek model
+   sysv <- rnorm(nsimu)
+   assetm <- matrix(rnorm(nsimu*nbonds), ncol=nsimu)
+   assetm <- t(rhos*sysv + t(rhosm*assetm))
+   lossv <- lgd*colSums(assetm < threshv)/nbonds
+   ## Calculate VaRs and CVaRs
+   vars <- quantile(lossv, probs=conv)
+   cvars <- sapply(vars, function(varisk) {
+     mean(lossv[lossv >= varisk])
+   }) ## end sapply
+   names(vars) <- conv
+   names(cvars) <- conv
+   c(vars, cvars)
+ } ## end calc_var
```

Standard Errors of VaR Using Bootstrap Simulation

The values of VaR and $CVaR$ produced by the function `calc_var()` are subject to uncertainty because they're calculated from a simulation.

We can calculate the standard errors of VaR and $CVaR$ by running the function `calc_var()` many times and repeating the simulation in a loop.

This bootstrap will only capture the uncertainty due to the finite number of trials in the simulation, but not due to the uncertainty of model parameters.

```
> # Define model parameters
> nbonds <- 300; nsimu <- 1000; lgd <- 0.4
> rho <- 0.2; rhos <- sqrt(rho); rhosm <- sqrt(1-rho)
> # Calculate default probabilities and thresholds
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> probv <- runif(nbonds, max=0.2)
> threshv <- qnorm(probv)
> conv <- seq(0.93, 0.99, 0.01)
> # Define number of bootstrap simulations
> nboot <- 500
> # Perform bootstrap of calc_var
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> bootd <- sapply(rep(lgd, nboot), calc_var,
+   threshv=threshv,
+   rhos=rhos, rhosm=rhosm,
+   nsimu=nsimu, conv=conv) ## end sapply
> bootd <- t(bootd)
> # Calculate standard errors of VaR and CVaR from bootd data
> varsd <- apply(bootd[, 1:7], MARGIN=2,
+   function(x) c(mean=mean(x), sd=sd(x)))
> cvarsd <- apply(bootd[, 8:14], MARGIN=2,
+   function(x) c(mean=mean(x), sd=sd(x)))
> # Scale the standard errors of VaRs and CVaRs
> varstds <- varsd[2, ]/varsd[1, ]
> cvarstds <- cvarsd[2, ]/cvarsd[1, ]
```

Standard Errors of VaR at High Confidence Levels

The standard errors of VaR and $CVaR$ are inversely proportional to square root of the number of loss events in the simulation, that exceed the VaR .

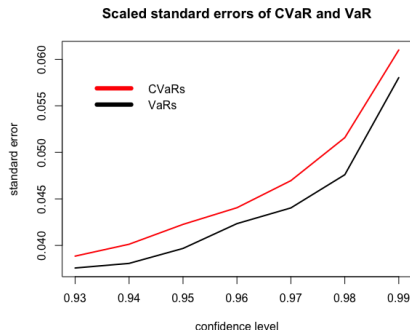
So the greater the number of loss events, the smaller the standard errors, and vice versa.

But as the confidence level increases, the VaR also increases, and the number of loss events decreases, causing larger standard errors.

So the as the confidence level increases, the standard errors of VaR and $CVaR$ also increase.

The *scaled* (relative) standard errors of VaR and $CVaR$ also increase with the confidence level, making them much less reliable at very high confidence levels.

The standard error of $CVaR$ is even greater than that of VaR .

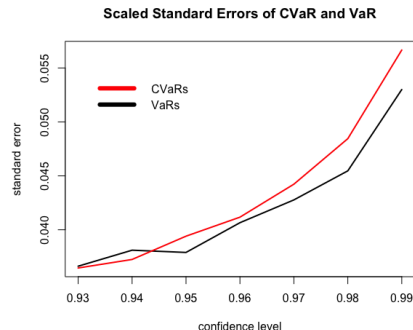


```
> # Plot the scaled standard errors of VaRs and CVaRs
> plot(x=names(varsds), y=varsds,
+      t="l", lwd=2, ylim=range(c(varsds, cvarsds)),
+      xlab="confidence level", ylab="standard error",
+      main="Scaled Standard Errors of CVaR and VaR")
> lines(x=names(cvarsds), y=cvarsds, lwd=2, col="red")
> legend(x="topleft", legend=c("CVaRs", "VaRs"), bty="n",
+       title=NULL, inset=0.05, cex=1.0, bg="white",
+       y.intersp=0.3, lwd=6, lty=1, col=c("red", "black"))
```

Standard Errors of VaR Using Parallel Bootstrap

The *scaled* standard errors of VaR and CVaR increase with the confidence level, making them much less reliable at very high confidence levels.

```
> library(parallel) ## load package parallel
> ncores <- detectCores() - 1 ## number of cores
> compclust <- makeCluster(ncores) ## Initialize compute cluster
> # Perform bootstrap of calc_var for Windows
> clusterSetRNGStream(compclust, 1121)
> bootd <- parLapply(compclust, rep(lgd, nboot),
+   fun=calc_var, threshv=threshv,
+   rhos=rhos, rhosm=rhosm,
+   nsimu=nsimu, conv=conv) ## end parLapply
> stopCluster(compclust) ## Stop R processes over cluster
> # Bootstrap under Mac-OSX or Linux
> bootd <- mclapply(rep(lgd, nboot),
+   FUN=calc_var, threshv=threshv,
+   rhos=rhos, rhosm=rhosm,
+   nsimu=nsimu, conv=conv) ## end mclapply
> bootd <- rutils::do_call(rbind, bootd)
> # Calculate standard errors of VaR and CVaR from bootd data
> varsd <- apply(bootd[, 1:7], MARGIN=2,
+   function(x) c(mean=mean(x), sd=sd(x)))
> cvarsd <- apply(bootd[, 8:14], MARGIN=2,
+   function(x) c(mean=mean(x), sd=sd(x)))
> # Scale the standard errors of VaRs and CVaRs
> varstds <- varsd[2, ]/varsd[1, ]
> cvarstds <- cvarsd[2, ]/cvarsd[1, ]
```



```
> # Plot the standard errors of VaRs and CVaRs
> plot(x=names(varstds), y=varstds, t="l", lwd=2,
+   ylim=range(c(varstds, cvarstds)),
+   xlab="confidence level", ylab="standard error",
+   main="Scaled Standard Errors of CVaR and VaR")
> lines(x=names(cvarstds), y=cvarstds, lwd=2, col="red")
> legend(x="topleft", legend=c("CVaRs", "VaRs"), bty="n",
+   title=NULL, inset=0.05, cex=1.0, bg="white",
+   y.intersp=0.3, lwd=6, lty=1, col=c("red", "black"))
```


Vasicek Model With Random Default Probabilities

The previous bootstrap only captured the uncertainty due to the finite simulation trials, but not due to the uncertainty of model parameters, such as the default probabilities and correlations.

The below function `calc_var()` can simulate the *Vasicek* model with random default probabilities.

```
> calc_var <- function(probv, ## Default probabilities
+   lgd=0.6, ## loss given default
+   rhos, rhosm, ## asset correlation
+   nsimu=1000, ## number of simulations
+   conv=seq(0.93, 0.99, 0.01) ## Confidence levels
+ ) {
+   ## Calculate random default thresholds
+   threshv <- qnorm(runif(1, min=0.5, max=1.5)*probv)
+   ## Simulate losses under the Vasicek model
+   nbonds <- NROW(probv)
+   sysv <- rnorm(nsimu)
+   assetm <- matrix(rnorm(nsimu*nbonds), ncol=nsimu)
+   assetm <- t(rhos*sysv + t(rhosm*assetm))
+   lossv <- lgd*colSums(assetm < threshv)/nbonds
+   ## Calculate VaRs and CVaRs
+   vars <- quantile(lossv, probs=conv)
+   cvars <- sapply(vars, function(varisk) {
+     mean(lossv[lossv >= varisk])
+   }) ## end sapply
+   names(vars) <- conv
+   names(cvars) <- conv
+   c(vars, cvars)
+ } ## end calc_var
```

Standard Errors Due to Random Default Probabilities

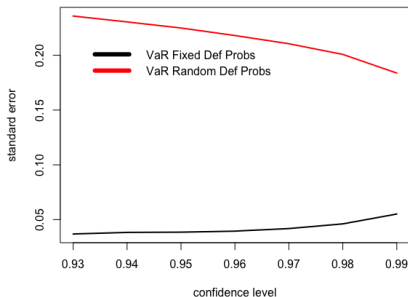
The greatest contribution to the standard errors of VaR and $CVaR$ is from the uncertainty of model parameters, such as the default probabilities, correlations, and loss severities.

For example, a 50% uncertainty in the default probabilities can produce a 20% uncertainty of the VaR .

When default probabilities are random, the standard error of the VaR decreases with higher confidence level, because the whole loss distribution shifts, so it has a larger effect at lower confidence levels.

```
> library(parallel) ## load package parallel
> ncores <- detectCores() - 1 ## number of cores
> compclust <- makeCluster(ncores) ## Initialize compute cluster
> # Perform bootstrap of calc_var for Windows
> clusterSetRNGStream(compclust, 1121)
> bootd <- parLapply(compclust, rep(lgd, nboot),
+   fun=calc_var, probv=probv, rhos=rhos, rhosm=rhosm,
+   nsimu=nsimu, conv=conv) ## end parLapply
> stopCluster(compclust) ## Stop R processes over cluster
> # Bootstrap under Mac-OSX or Linux
> bootd <- mclapply(rep(lgd, nboot),
+   FUN=calc_var, probv=probv, rhos=rhos, rhosm=rhosm,
+   nsimu=nsimu, conv=conv) ## end mclapply
> bootd <- rutils::do_call(rbind, bootd)
> # Calculate standard errors of VaR and CVaR from bootd data
> varsd <- apply(bootd[, 1:7], MARGIN=2,
+   function(x) c(mean=mean(x), sd=sd(x)))
> cvarsd <- apply(bootd[, 8:14], MARGIN=2,
+   function(x) c(mean=mean(x), sd=sd(x)))
> # Scale the standard errors of VaRs and CVaRs
> varsd_s <- varsd[2, ]/varsd[1, ]
> cvarsd_s <- cvarsd[2, ]/cvarsd[1, ]
```

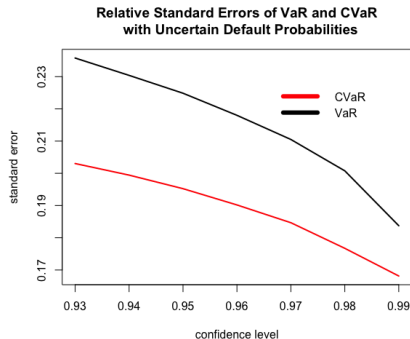
**Standard Errors of VaR
with Random Default Probabilities**



```
> # Plot the standard errors of VaRs under random default probabilities
> plot(x=colnames(varsd), y=varsds, t="l",
+   col="black", lwd=2, ylim=range(c(varsds, varsdsu)),
+   xlab="confidence level", ylab="standard error",
+   main="Standard Errors of VaR
+   with Random Default Probabilities")
> lines(x=colnames(varsd), y=varsdsu, lwd=2, col="red")
> legend(x="topleft",
+   legend=c("VaR Fixed Def Probs", "VaR Random Def Probs"),
+   bty="n", title=NULL, inset=0.05, cex=1.0, bg="white",
+   y.intersp=0.3, lwd=6, lty=1, col=c("black", "red"))
```

Relative Errors Due to Random Default Probabilities

When default probabilities are random, the *scaled* (relative) standard errors of VaR and $CVaR$ decrease with higher confidence level, because the whole loss distribution shifts, so it has a larger effect at lower confidence levels.



```
> # Plot the standard errors of VaRs and CVaRs
> plot(x=colnames(wards), y=varsdsu, t="l", lwd=2,
+      ylim=range(c(varsdsu, cvarsdsu)),
+      xlab="confidence level", ylab="standard error",
+      main="Relative Standard Errors of VaR and CVaR
+      with Random Default Probabilities")
> lines(x=colnames(wards), y=cvarsdsu, lwd=2, col="red")
> legend(x="topright", legend=c("CVaR", "VaR"), bty="n",
+       title=NULL, inset=0.05, cex=1.0, bg="white",
+       y.intersp=0.3, lwd=6, lty=1, col=c("red", "black"))
```

Model Risk of Credit Portfolio Models

Credit portfolio models are subject to very significant *model risk* due to the uncertainties of model parameters, such as the default probabilities, correlations, and loss severities.

Model risk is the risk of incorrect model predictions due to incorrect model specification, and due to incorrect model parameters.

Jon Danielsson at the London School of Economics (LSE) has studied the model risk of VaR and $CVaR$ in: [Why Risk is So Hard to Measure](#), and in [Model Risk of Risk Models](#).

Jon Danielsson has pointed out that there's not enough historical data to be able to accurately calculate the credit model parameters.

Jon Danielsson and Chen Zhou have demonstrated that accurately estimating $CVaR$ at 5% confidence [would require decades of price history](#), something that simply doesn't exist for many assets.

depr: Simulating Credit Portfolios Using Importance Sampling

It's not possible to apply importance sampling to the credit loss distribution because the distribution is not known.

Importance sampling can be used to improve the accuracy of VaR and CVaR calculations for credit portfolios.

```
> #####
> # Summary: Calculate the VaR and CVaR of a credit
> # portfolio using importance sampling.
>
> ## Run the setup code below.
>
> # Define model parameters
> nbonds <- 10; nsimu <- 1000; lgd <- 0.4
> # Define correlation parameters
> rho <- 0.2; rhos <- sqrt(rho); rhosm <- sqrt(1-rho)
> # Calculate default probabilities and thresholds
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> probv <- runif(nbonds, max=0.2)
> threshv <- qnorm(probv)
> # Simulate losses under the Vasicek model
> sysv <- rnorm(nsimu)
> assetm <- matrix(rnorm(nsimu*nbonds), ncol=nsimu)
> assetm <- t(rhos*sysv + t(rhosm*assetm))
> lossv <- lgd*colSums(assetm < threshv)/nbonds
>
> # Calculate VaR from confidence level
> confl <- 0.95
> varisk <- quantile(lossv, confl)
>
> cumprob <- (1:nsimu)/nsimu
> sort(lossv)[findInterval(confl, cumprob)]
>
> # Calculate the CVaR as the mean losses in excess of VaR
> cvar <- mean(lossv[lossv > varisk])
>
>
> # Apply importance sampling
> lambdaf <- (-0.1)
> assett <- assetm + lambdaf
> weightv <- exp(-lambdaf*assetm + lambdaf^2/2)
>
> losst <- lgd*colSums(assett < threshv)/nbonds
> lossst <- matrixStats::colMeans(assett)
```

draft: Standard Errors of VaR Using Bootstrap Simulation

The values of VaR and $CVaR$ produced by the function `calc_var()` are subject to uncertainty because they're calculated from a simulation.

We can calculate the standard errors of VaR and $CVaR$ by running the function `calc_var()` many times and repeating the simulation in a loop.

This bootstrap will only capture the uncertainty due to the finite number of trials in the simulation, but not due to the uncertainty of model parameters.

```
> # Define model parameters
> nbonds <- 300; nsimu <- 1000; lgd <- 0.4
> rho <- 0.2; rhos <- sqrt(rho); rhosm <- sqrt(1-rho)
> # Calculate default probabilities and thresholds
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> probv <- runif(nbonds, max=0.2)
> threshv <- qnorm(probv)
> # Define number of bootstrap simulations
> nboot <- 500
> # Perform bootstrap of calc_var
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> bootd <- sapply(rep(lgd, nboot),
+   calc_var,
+   threshv=threshv,
+   rhos=rhos, rhosm=rhosm,
+   nsimu=nsimu, conv=conv) ## end sapply
> bootd <- t(bootd)
> # Calculate standard errors of VaR and CVaR from bootd data
> varsd <- apply(bootd[, 1:7], MARGIN=2,
+   function(x) c(mean=mean(x), sd=sd(x)))
> cvarsd <- apply(bootd[, 8:14], MARGIN=2,
+   function(x) c(mean=mean(x), sd=sd(x)))
> # Scale the standard errors of VaRs and CVaRs
> varsd[2, ] <- varsd[2, ]/varsd[1, ]
> cvarsd[2, ] <- cvarsd[2, ]/cvarsd[1, ]
```